

Sprint Report 1

Unit code: COS40005

Unit Name: Computing Technology Project A

Submission date: 19/04/2026

P54 (Afzal) Embodied Multimodal LLM for Industrial Task Planning

Team Members :

- 1) Ved Jay Makhijani — 104762184
- 2) Minh Hoang Duong — 104487115
- 3) Kaveesha Dharmadasa — 105271678
- 4) Dinith Thejana — 105231766
- 5) Lakshit Bansal — 105028858

ACKNOWLEDGMENT OF COUNTRY

Each team member identifies the Traditional Owners of the land they lived on while completing this work (if living in Australia)...

Kumarawadu Dinith Thejana:

I, Kumarawadu Dinith Thejana, acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne Australian campuses are located in Melbourne east and outer-east, and pay our respect to their Elders past, present and emerging, and recognise their enduring connection to Country, culture and community. I am committed to honouring this land and the peoples whose knowledge and traditions have shaped it since time immemorial.

Minh Hoang Duong:

I, Minh Hoang Duong, acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne Australian campuses are located in Melbourne east and outer-east, and pay our respect to their Elders past, present and emerging, and recognise their enduring connection to Country, culture and community. I am committed to honouring this land and the peoples whose knowledge and traditions have shaped it since time immemorial.

Kaveesha Dharmadasa

I Kaveesha Dharmadasa acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne Australian campuses are located in Melbourne east and outer-east, and pay our respect to their Elders past, present and emerging, and recognise their enduring connection to Country, culture and community. I am committed to honouring this land and the peoples whose knowledge and traditions have shaped it since time immemorial.

Ved Jay Makhijani

I, Ved Jay Makhijani, respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

Lakshit Bansal

I, Lakshit Bansal, acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne Australian campuses are located in Melbourne east and outer-east, and pay our respect to their Elders past, present and emerging, and recognise their enduring connection to Country, culture and community. I am committed to honouring this land and the peoples whose knowledge and

traditions have shaped it since time immemorial

CONTRIBUTION SUMMARY (THIS SPRINT)

All team members should complete the following table together. You can provide a yes/no response or a brief comment where appropriate.

	Contribution	Initiatives	Communication					Respect
Team Member	Finished tasks in time or on time with an acceptable quality	Self-assigned tasks and proactively found solutions to problems	Attended all team meetings	Attended all supervisory meetings	Attended all client meetings	Replied every time within an acceptable timeframe	Kept the team updated about the status	The student respected others and listened to different opinions
Ved Jay Makhijani - 104762184	Yes – Completed PB6 (LLM instruction parser) fully: schema, prompts, edge cases, parser, 24/24 unit tests passing, and interactive demo working end-to-end, and Comparative Analysis of all models.	Yes – Self-selected PB6, designed the output schema independently, established shared interface contract for team, resolved Python 3.14/LangChain compatibility issues	Yes	Yes	Yes	Yes – Responded promptly via Teams and kept team informed of module status	Yes – Shared schema.py interface with team and pushed all code to GitHub with clear commit messages	Yes – Collaborative, supported teammates and listened to design feedback during meetings

Minh Hoang Duong - 104487115	Yes -- Completed full refactoring of the LLM module from two duplicated folder structures (llm_module_gemini_files, llm_module_openAI) into a single modular llm_backend with four pluggable backends (OpenAI, Gemini, DeepSeek, HuggingFace local).	Yes -- Self-identified the architectural problem of duplicated module folders, independently designed the pluggable backend factory pattern Researched and integrated DeepSeek and HuggingFace as new backends not originally in scope, and proactively resolved all cross-platform compatibility	Yes	Yes	Yes	Yes - Responded promptly via Teams, actively sending update messages and asking permission	Yes -- Pushed all refactored code to GitHub with a clear commit message. Shared updated schema.py interface and .env.example with team.	Yes – Collaborative, supported teammates and listened to design feedback during meetings
Kaveesha Dharmadasa 105271678	Yes – Completed key tasks including vision framework research, JSON design, and architecture documentation on time with good quality	Yes – Took initiative in researching YOLO vs OpenCV, designed scene abstraction structure, and recommended LLM pipeline	Yes	Yes	Yes	Yes – Responded actively in team discussions and updates	Yes – Shared documentation and updates via repository and discussions	Yes – Maintained good collaboration and considered team input

Dinith Thejana 105231766	Yes – Completed key technical tasks including development of vision output module, scene representation, and spatial relationship computation with correct JSON outputs.	Yes - took initiative to implement the full vision pipeline (detection → scene → relationships), structured JSON design, and tested outputs in VS Code.	Yes	Yes	Yes	Yes – Responded actively in team discussions and updates	Yes – Shared progress, code outputs, and GitHub updates with the team	Yes- Maintain good collaboration,s upport team member, and respected different ideas
Lakshit Bansal 105028858	Yes - Completed all the tasks in relation to development of simulation environment and computer vision from scratch	Yes - Did all the debugging and research about running the simulation on macOS and also added the functionality of of being able to use gemini API for the llm module.	Yes	Yes	Yes	Yes – Responded actively in team discussions and updates	Yes – Shared progress, code outputs, and GitHub updates with the team	Yes- Maintain good collaboration,s upport team member, and respected different ideas

CONTRIBUTION DETAILS (THIS SPRINT)

In this section, every member of the team should write a summary of what they have contributed during this sprint. All contributions must be supported by evidence, such as the number of GitHub commits, a screenshot of developed screens, a summary/outcome of research, etc.

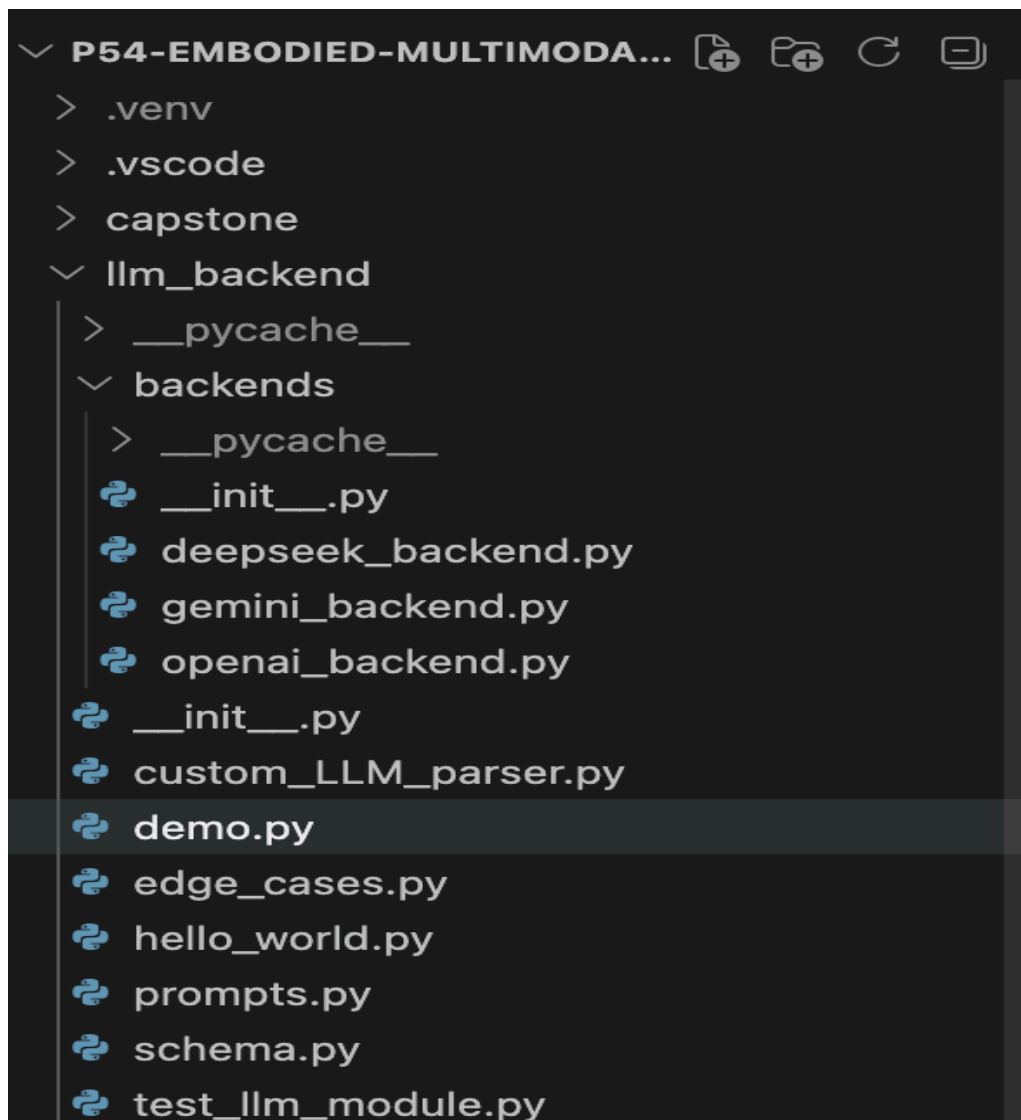
TEAM MEMBER: VED JAY MAKHIJANI (104762184)

During Sprint 1, my primary responsibility was PB6 — the LLM Instruction Parser Module. This module is the entry point of the entire AI pipeline, responsible for converting natural language user instructions into structured JSON that all downstream modules (scene representation, task planner, execution) consume. I completed this ticket in full and it is the most technically complex individual deliverable of Sprint 1.

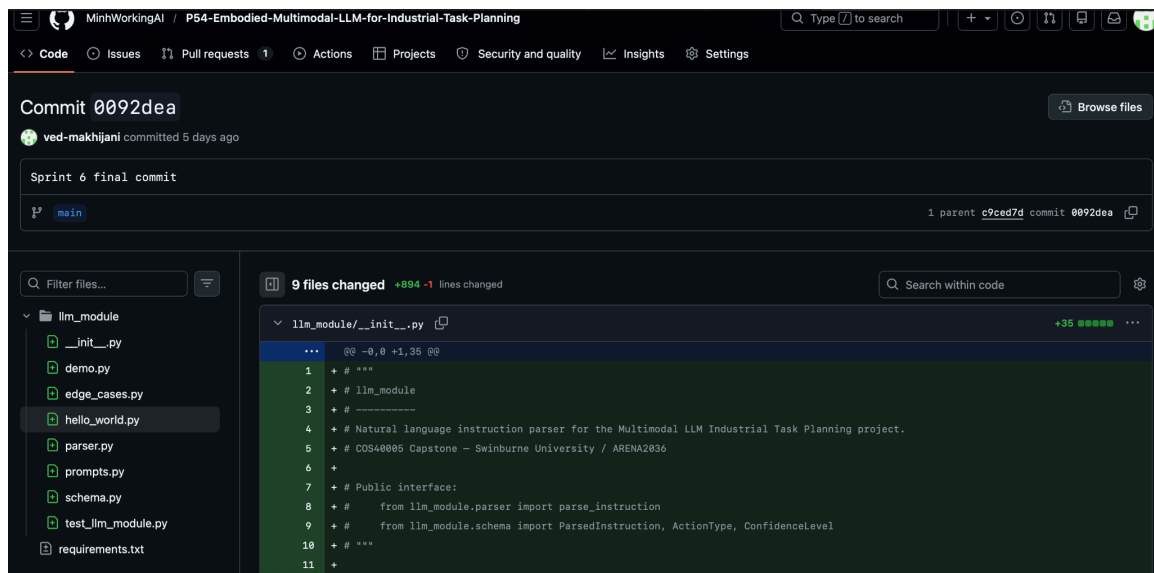
Task 1: LLM Module Setup and Project Structure (PB3 – LLM research slice / PB6)

I selected GPT-4o via the OpenAI API with LangChain as the orchestration framework, based on a comparative evaluation of available models against criteria of JSON output reliability, latency, and integration ease. I initialised the `llm_module/` folder with a clean modular structure: `schema.py` (Pydantic output model), `parser.py` (LangChain chain), `prompts.py` (system prompt and few-shot examples), `edge_cases.py` (pre/post processing), `hello_world.py` (connection test), `demo.py` (Sprint 1 demo script), and `__init__.py` (package entry point). I also set up the project's GitHub repository folder structure and configured environment variable loading via `python-dotenv`.

1) *llm_module/ folder structure in VS Code file explorer showing all files*



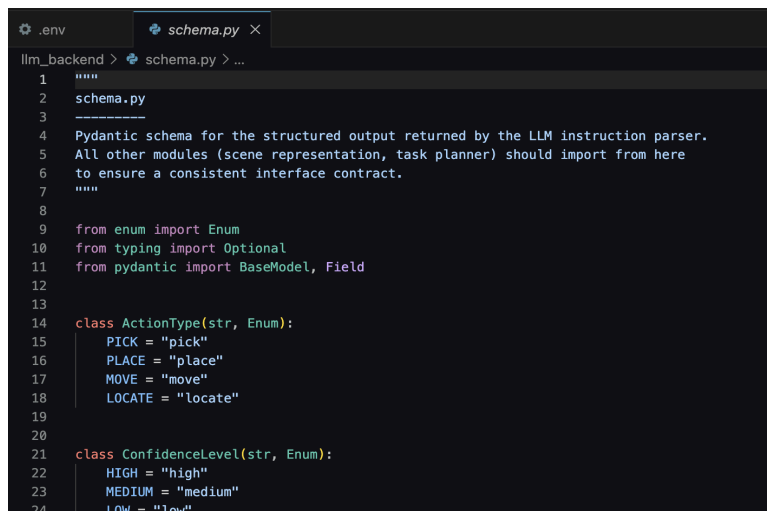
2) *GitHub repository showing the initial commit with llm_module folder and all files*



Task 2: Pydantic Output Schema Design (PB6)

I designed the ParsedInstruction schema using Pydantic v2, which defines the strict contract between my module and all downstream consumers (PB5 scene representation, PB7 task planner). The schema enforces four allowed ActionTypes (pick, place, move, locate) via an Enum, three ConfidenceLevels (high, medium, low), and fields for object_target, destination, spatial_relation, raw_instruction, and notes. I documented this schema and shared it with the team as the formal interface agreement — no other module is permitted to modify it without team consensus.

- 3) *schema.py code showing the ParsedInstruction Pydantic model, ActionType enum, and ConfidenceLevel enum*



Task 3: Prompt Engineering and Few-Shot Examples (PB6)

I wrote and iteratively refined the system prompt in `prompts.py`, incorporating six few-shot examples covering: simple pick instructions, place with destination and spatial relation, synonym mapping (“grab” → pick), ambiguous instructions (low confidence), and multi-part instructions. Temperature was set to 0.0 to maximise output determinism. The prompt enforces that the model must always return valid JSON matching the schema, explicitly lists the four allowed actions, and requires that ambiguous instructions be flagged with a confidence of “low” and an explanatory note.

4) *prompts.py showing the system prompt template and the six few-shot example input/output pairs*

```
# -- System prompt -----
SYSTEM_PROMPT_TEMPLATE = """
You are an instruction parser for an intelligent industrial robot operating \
in a simulated workspace.

Your job is to read a natural language task instruction and extract structured \
information from it as JSON.

ALLOWED ACTIONS (use only these):
- pick   : picking up / grabbing an object
- place  : putting / dropping / placing an object somewhere
- move   : moving an object to a location
- locate : finding / locating an object

WORKSPACE OBJECTS: coloured blocks (red, blue, green, yellow), trays (left tray, \
right tray), workstations.

RULES:
1. Always return valid JSON matching the schema exactly.
2. If an action word is not in the allowed list, map it to the closest valid action \
   and explain in notes.
3. If the instruction is ambiguous or vague, set confidence to "low" and explain in notes.
4. If no destination is mentioned, set destination to null.
5. If no spatial relation is mentioned, set spatial_relation to null.
6. Never invent objects or locations not mentioned in the instruction.
7. Always preserve the original instruction in raw_instruction exactly as given.
8. If the object or destination is completely unclear, set them to "unknown".
```

```

FEW_SHOT_EXAMPLES = [
    {
        "instruction": "pick up the red block",
        "output": {
            "action": "pick",
            "object_target": "red block",
            "destination": None,
            "spatial_relation": None,
            "confidence": "high",
            "raw_instruction": "pick up the red block",
            "notes": None
        }
    },
    {
        "instruction": "place the blue cube in the left tray",
        "output": {
            "action": "place",
            "object_target": "blue cube",
            "destination": "left tray",
            "spatial_relation": "in",
            "confidence": "high",
            "raw_instruction": "place the blue cube in the left tray",
            "notes": None
        }
    },
]

```

```

{
  "instruction": "move the green block to the right of the workstation",
  "output": {
    "action": "move",
    "object_target": "green block",
    "destination": "workstation",
    "spatial_relation": "right of",
    "confidence": "high",
    "raw_instruction": "move the green block to the right of the workstation",
    "notes": None
  }
},
{
  "instruction": "where is the yellow block",
  "output": {
    "action": "locate",
    "object_target": "yellow block",
    "destination": None,
    "spatial_relation": None,
    "confidence": "high",
    "raw_instruction": "where is the yellow block",
    "notes": None
  }
},

```

```

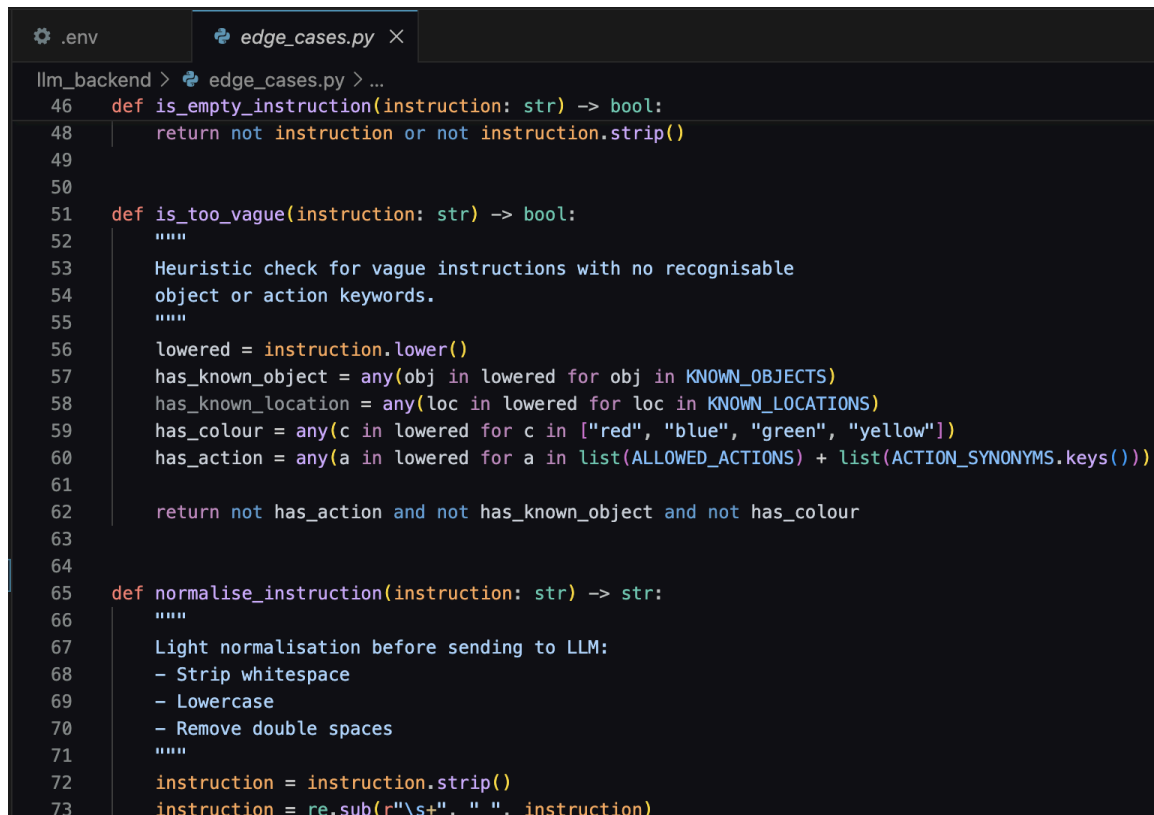
  "instruction": "put that thing over there",
  "output": {
    "action": "move",
    "object_target": "unknown",
    "destination": "unknown",
    "spatial_relation": None,
    "confidence": "low",
    "raw_instruction": "put that thing over there",
    "notes": "Instruction is too vague. Object and destination are not specified."
  }
},
{
  "instruction": "grab the red block and drop it near the blue tray",
  "output": {
    "action": "pick",
    "object_target": "red block",
    "destination": "blue tray",
    "spatial_relation": "near",
    "confidence": "high",
    "raw_instruction": "grab the red block and drop it near the blue tray",
    "notes": "Mapped 'grab' to 'pick' and 'drop' to 'place' as closest valid actions."
  }
},

```

Task 4: Edge Case Handling (PB6)

I implemented `edge_cases.py` to handle all abnormal inputs before and after the LLM call. Pre-processing functions detect empty/whitespace instructions (raises `ValueError` immediately), vague instructions with no recognisable objects or actions (short-circuits the API call and returns a low-confidence result without wasting tokens), and normalise whitespace. Post-processing functions validate the returned `ParsedInstruction`, downgrade confidence to low if `object_target` or `destination` is “unknown”, and append explanatory notes. An action synonym dictionary (grab→pick, drop→place, find→locate, etc.) guides the LLM toward valid actions.

- 5) *edge_cases.py* showing the `is_too_vague()`, `normalise_instruction()`, and `validate_parsed_result()` functions



```
.env  edge_cases.py X
llm_backend > edge_cases.py > ...
46 def is_empty_instruction(instruction: str) -> bool:
47     return not instruction or not instruction.strip()
48
49
50
51 def is_too_vague(instruction: str) -> bool:
52     """
53     Heuristic check for vague instructions with no recognisable
54     object or action keywords.
55     """
56     lowered = instruction.lower()
57     has_known_object = any(obj in lowered for obj in KNOWN_OBJECTS)
58     has_known_location = any(loc in lowered for loc in KNOWN_LOCATIONS)
59     has_colour = any(c in lowered for c in ["red", "blue", "green", "yellow"])
60     has_action = any(a in lowered for a in list(ALLOWED_ACTIONS) + list(ACTION_SYNONYMS.keys()))
61
62     return not has_action and not has_known_object and not has_colour
63
64
65 def normalise_instruction(instruction: str) -> str:
66     """
67     Light normalisation before sending to LLM:
68     - Strip whitespace
69     - Lowercase
70     - Remove double spaces
71     """
72     instruction = instruction.strip()
73     instruction = re.sub(r"\s+", " ", instruction)
```

Task 5: Unit Test Suite – 24/24 Passing (PB6)

I wrote a comprehensive test suite in `test_llm_module.py` with 35 tests organised into three classes: `TestSchema` (6 tests — validates Pydantic model accepts valid inputs and rejects invalid action types), `TestEdgeCases` (18 tests — covers all pre/post processing functions without any API calls), and `TestParserIntegration` (11 tests — real LLM calls, marked with `@pytest.mark.integration` and skipped without API credits). Running `pytest test_llm_module.py -v -m "not integration"` yields 24 passed with 0 failures. This means all core module logic can be verified by any team member at any time, even without an API key.

- 6) *Terminal showing pytest output: 24 passed, 0 failed, 11 deselected (integration tests skipped)]*

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) ved@Veds-MacBook-Air P54-Embodied-Multimodal-LLM-for-Industrial-Task-Planning % python3 -m pytest llm_backend/test_llm_module.py -v -m "not integration"
llm_backend/test_llm_module.py::TestSchema::test_valid_minimal_instruction PASSED [ 7%]
llm_backend/test_llm_module.py::TestSchema::test_action_types_are_valid PASSED [ 10%]
llm_backend/test_llm_module.py::TestSchema::test_confidence_levels_are_valid PASSED [ 14%]
llm_backend/test_llm_module.py::TestSchema::test_invalid_action_raises PASSED [ 17%]
llm_backend/test_llm_module.py::TestSchema::test_model_dump_returns_dict PASSED [ 21%]
llm_backend/test_llm_module.py::TestEdgeCases::test_empty_string_is_empty PASSED [ 25%]
llm_backend/test_llm_module.py::TestEdgeCases::test_whitespace_only_is_empty PASSED [ 28%]
llm_backend/test_llm_module.py::TestEdgeCases::test_none_is_empty PASSED [ 32%]
llm_backend/test_llm_module.py::TestEdgeCases::test_valid_instruction_not_empty PASSED [ 35%]
llm_backend/test_llm_module.py::TestEdgeCases::test_vague_instruction_detected PASSED [ 39%]
llm_backend/test_llm_module.py::TestEdgeCases::test_clear_instruction_not_vague PASSED [ 42%]
llm_backend/test_llm_module.py::TestEdgeCases::test_colour_mention_not_vague PASSED [ 46%]
llm_backend/test_llm_module.py::TestEdgeCases::test_action_mention_not_vague PASSED [ 50%]
llm_backend/test_llm_module.py::TestEdgeCases::test_strips_whitespace PASSED [ 53%]
llm_backend/test_llm_module.py::TestEdgeCases::test_collapses_double_spaces PASSED [ 57%]
llm_backend/test_llm_module.py::TestEdgeCases::test_preserves_content PASSED [ 60%]
llm_backend/test_llm_module.py::TestEdgeCases::test_vague_result_has_low_confidence PASSED [ 64%]
llm_backend/test_llm_module.py::TestEdgeCases::test_vague_result_preserves_instruction PASSED [ 67%]
llm_backend/test_llm_module.py::TestEdgeCases::test_vague_result_has_notes PASSED [ 71%]
llm_backend/test_llm_module.py::TestEdgeCases::test_vague_result_object_is_unknown PASSED [ 75%]
llm_backend/test_llm_module.py::TestEdgeCases::test_unknown_object_downgrades_confidence PASSED [ 78%]
llm_backend/test_llm_module.py::TestEdgeCases::test_known_object_keeps_confidence PASSED [ 82%]
llm_backend/test_llm_module.py::TestEdgeCases::test_unknown_destination_adds_notes PASSED [ 85%]
llm_backend/test_llm_module.py::TestBackendFactory::test_invalid_backend_raises PASSED [ 89%]
llm_backend/test_llm_module.py::TestBackendFactory::test_missing_openai_key_raises PASSED [ 92%]
llm_backend/test_llm_module.py::TestBackendFactory::test_missing_gemini_key_raises PASSED [ 96%]
```

Task 6: Demo Script and End-to-End Verification (PB6)

I built demo.py with two modes: a preset demo (python demo.py) that runs six representative instructions and prints colour-coded JSON results, and an interactive mode (python demo.py --interactive) for live demonstration. I verified the module end-to-end against the OpenAI API, confirming that all four instruction types (simple, spatial, synonym, ambiguous) were parsed correctly on the first attempt with correct action, object, destination, spatial relation and confidence fields returned.

7) Terminal showing demo.py --interactive output

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(.venv) ved@Veds-MacBook-Air P54-Embodied-Multimodal-LLM-for-Industrial-Task-Planning % python3 -m llm_backend.demo --interactive
Type an instruction and press Enter.
Type 'quit' to exit.
=====
Instruction: grab the red ball
2026-04-18 00:38:50,886 [INFO] Parsing: 'grab the red ball' via openai backend
2026-04-18 00:38:52,487 [INFO] HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 200 OK"
2026-04-18 00:38:52,527 [INFO] Parsed successfully on attempt 1: action=<ActionType.PICK: 'pick'> object_target='red ball' destination=None spatial_relation=None confidence=<ConfidenceLevel.HIGH: 'high'> raw_instruction='grab the red ball' notes='Mapped 'grab' to 'pick' as the closest valid action.'
=====
Instruction: grab the red ball
Action: pick
Object: red ball
Destination: -
Spatial: -
Confidence: high
Notes: Mapped 'grab' to 'pick' as the closest valid action.
=====
Instruction: deposit the black cube
2026-04-18 00:39:12,536 [INFO] Parsing: 'deposit the black cube' via openai backend
2026-04-18 00:39:15,018 [INFO] HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 200 OK"
2026-04-18 00:39:15,020 [INFO] Parsed successfully on attempt 1: action=<ActionType.PLACE: 'place'> object_target='black cube' destination=None spatial_relation=None confidence=<ConfidenceLevel.LOW: 'low'> raw_instruction='deposit the black cube' notes='Mapped 'deposit' to 'place' as the closest valid action. The destination is not specified and the object 'black cube' is not mentioned in the workspace objects.'
=====
Instruction: deposit the black cube
Action: place
Object: black cube
Destination: -
Spatial: -
Confidence: low
Notes: Mapped 'deposit' to 'place' as the closest valid action. The destination is not specified and the object 'black cube' is not mentioned in the workspace objects.
=====
Instruction: find the red ball
2026-04-18 00:39:32,331 [INFO] Parsing: 'find the red ball' via openai backend
2026-04-18 00:39:33,998 [INFO] HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 200 OK"
2026-04-18 00:39:33,998 [INFO] Parsed successfully on attempt 1: action=<ActionType.LOCATE: 'locate'> object_target='red ball' destination=None spatial_relation=None confidence=<ConfidenceLevel.HIGH: 'high'> raw_instruction='find the red ball' notes='Mapped 'find' to 'locate' as the closest valid action.'
=====
Instruction: find the red ball
Action: locate
Object: red ball
Destination: -
Spatial: -
Confidence: high
Notes: Mapped 'find' to 'locate' as the closest valid action.
=====
Instruction: quit
Goodbye!
(.venv) ved@Veds-MacBook-Air P54-Embodied-Multimodal-LLM-for-Industrial-Task-Planning %
```

Task 7: Comparative LLM Evaluation Results

The table below summarizes the performance of the three evaluated models across 25 test instructions.

Metric	GPT-4o	DeepSeek	Gemini
Instruction Accuracy (%)	76.0	76.0	56.0
Parse Success Rate (%)	100.0	100.0	84.0
Action Accuracy (%)	100.0	100.0	76.0
Object Accuracy (%)	96.0	96.0	80.0
Destination Accuracy (%)	96.0	96.0	80.0
Spatial Accuracy (%)	84.0	84.0	72.0
Avg Latency (ms)	1,149.6	3,805.9	10,475.8
Error Rate (%)	0.0	0.0	16.0

Key Findings by Category (Instruction Accuracy %)

- **Simple Instructions:** All three models (GPT-4o, DeepSeek, and Gemini) achieved **100.0%** accuracy for basic single-action commands.
- **Spatial Reasoning:** GPT-4o and DeepSeek maintained **100.0%** accuracy, while Gemini dropped to **80.0%**.
- **Edge Cases:** GPT-4o and DeepSeek both achieved **75.0%** accuracy. In contrast, Gemini failed significantly in this category with a **0.0% success rate and a 100.0% error rate**.
- **Multi-Step Tasks:** Complexity impacted all models, with GPT-4o and DeepSeek at **66.7%** and Gemini at **33.3%**.

Analysis and Recommendations for Sprint 1

- **Model Selection:** GPT-4o is the recommended primary model due to its high accuracy and superior speed. It recorded the lowest average latency (**1,149.6 ms**), which is critical for real-time robotic execution.
- **Gemini Performance Issues:** Gemini's high average latency (**10,475.8 ms**) and failure to handle edge cases make it less suitable for the current pipeline requirements.
- **Latency Impact:** The significant latency in Gemini (peaking at over 22 seconds in some cases) could delay the robotics module testing process, similar to the hardware compatibility issues already identified.
- **Future Plan:** While GPT-4o is currently the most reliable for complex reasoning, the team will continue to investigate local, open-source models as backups to adhere to Australian Privacy Principles (APPs) for future real-world data handling.

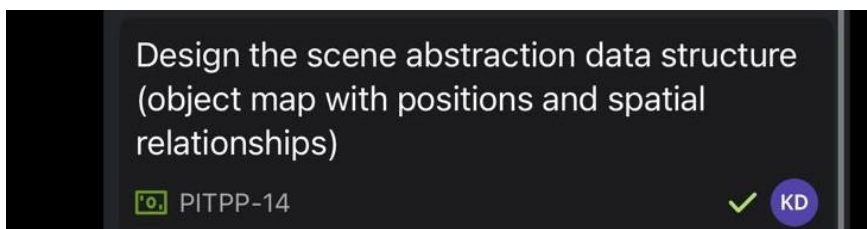
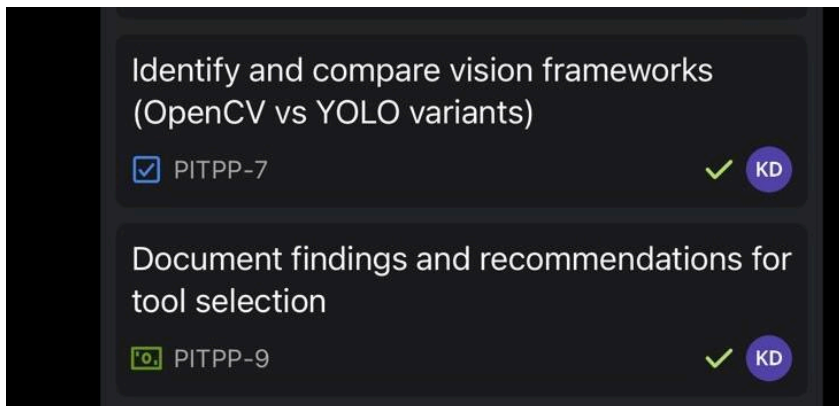
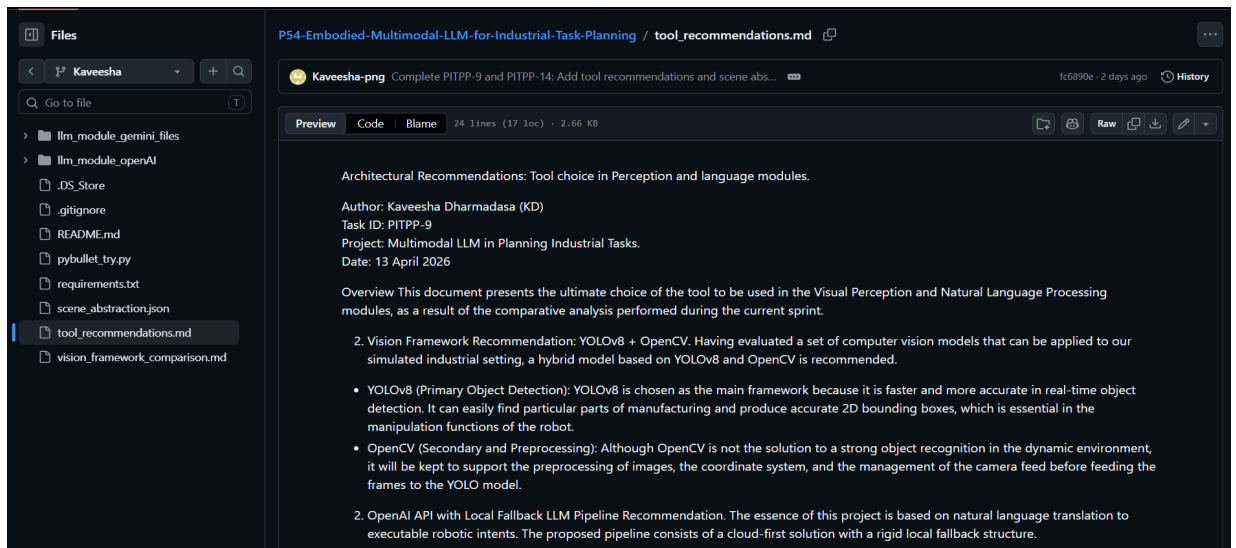
TEAM MEMBER: KAVEESHA DHARMADASA. (105271678)

This sprint involved my main technical contribution that included filling the gap between the initial research phase and the system architecture design. I managed to accomplish three significant tasks, which are to document the tool choice of the perception, Find and Compare Vision Frameworks and language modules and to design the core scene abstraction data structure.

1. Document Findings and Recommendations on Tool Selection (Task ID: PITPP-9)

According to the comparative analysis that the team did, I finalized and documented the architectural recommendations to the project:

- **Vision Framework Recommendation (YOLOv8 + OpenCV):** I suggested using YOLOv8 as the main object detector because of its high accuracy and speed in object recognition and generating the manufacturing components and bounding boxes in the correct position. OpenCV will be kept as a secondary coordinate mapping tool and pre-processing camera feed and sending it to YOLO.
- **LLM Pipeline Recommendation (OpenAI API with Local Fallback):** In the first prototype, I advised to use the OpenAI API (GPT-4) because it is the most reliable zero-shot intent extraction in complex reasoning (ReAct prompting) at the moment. But to adhere to the Australian Privacy Principles as described in our ethical considerations, I wrote that a local, open-source model should be implemented as a backup in subsequent sprints to manage potentially sensitive factory instructions.

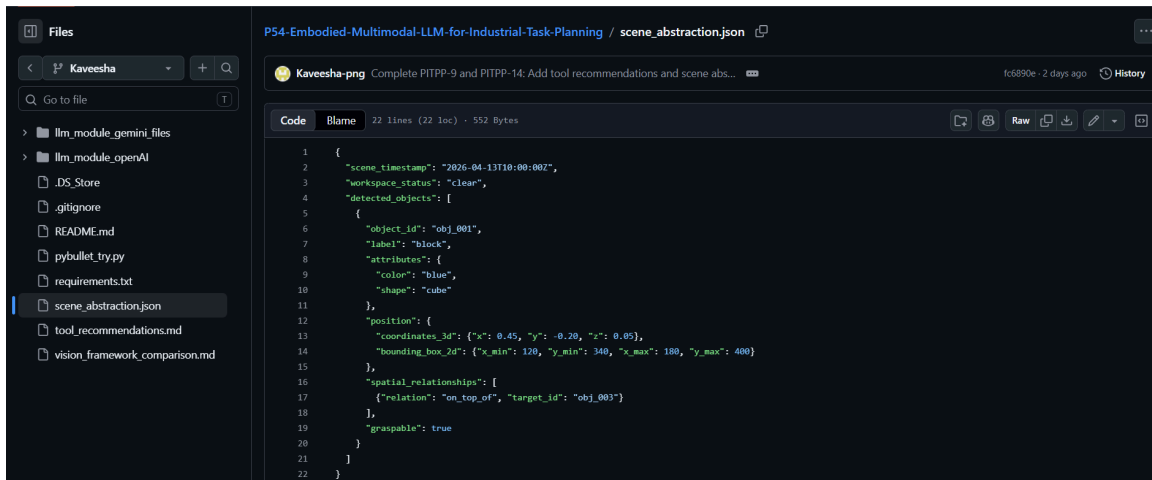


2. Design the Scene Abstraction Data Structure (Task ID: PITPP-14)

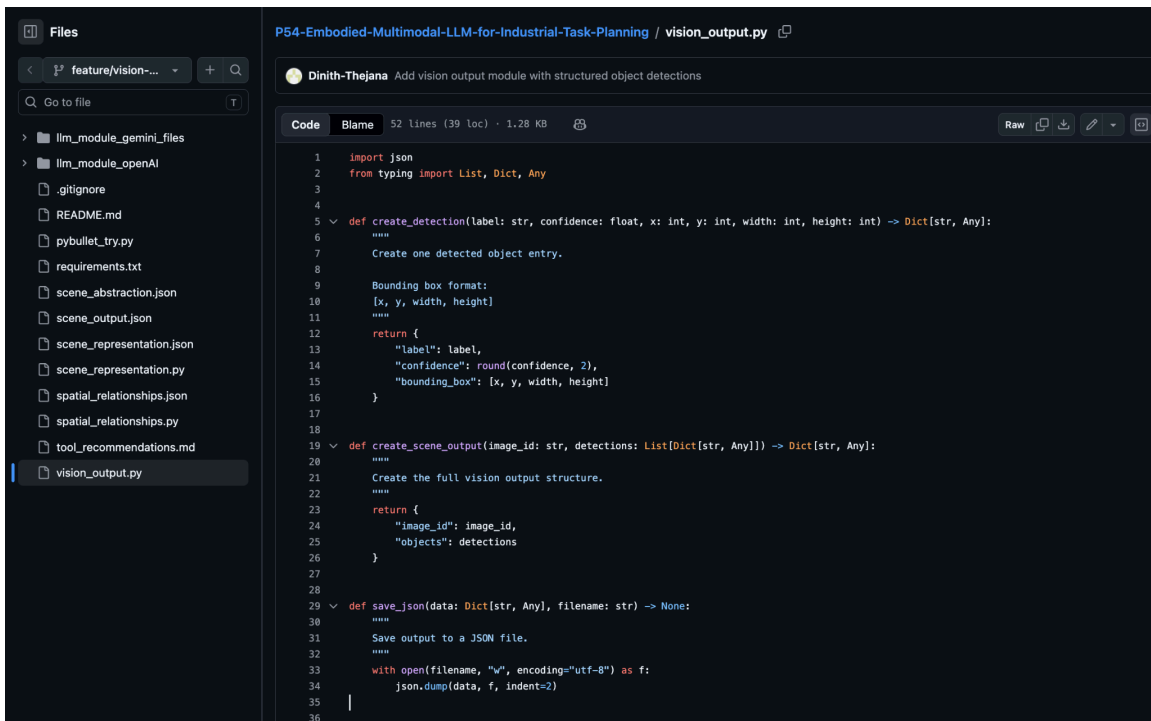
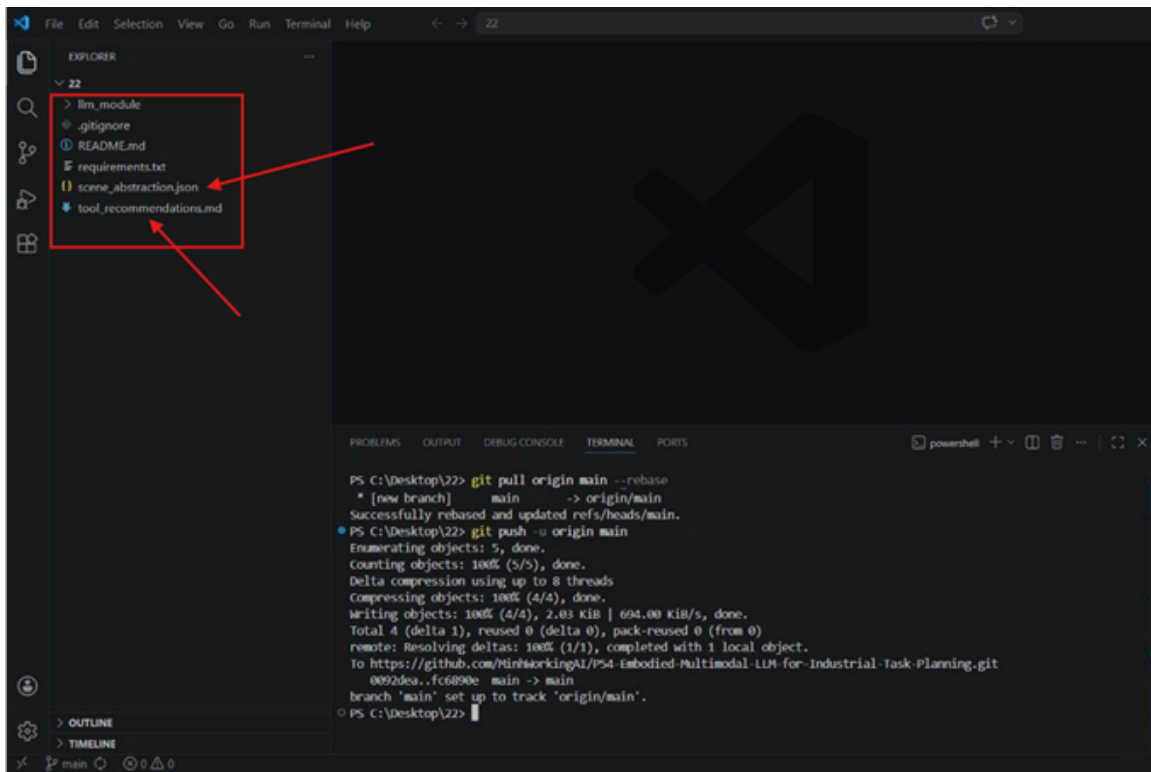
To allow the LLM Central Task Planner to get familiar with the physical environment, I developed a common JSON data format. This architecture is the crucial agreement between the Visual Perception module (object-detecting) and the Task Planner (action-generating). The structure generalizes the simulation environment into an object map with object IDs, physical attributes, 3D spatial coordinates and contextual relationships.



```
1 {
2   "scene_timestamp": "2026-04-13T10:00:00Z",
3   "workspace_status": "clear",
4   "detected_objects": [
5     {
6       "object_id": "obj_001",
7       "label": "block",
8       "attributes": {
9         "color": "blue",
10        "shape": "cube"
11      },
12      "position": {
13        "coordinates_3d": {"x": 0.45, "y": -0.20, "z": 0.05},
14        "bounding_box_2d": {"x_min": 120, "y_min": 340, "x_max": 180, "y_max": 400}
15      },
16      "spatial_relationships": [
17        {"relation": "on_top_of", "target_id": "obj_003"}
18      ],
19      "graspable": true
20    }
21  ]
22 }
```



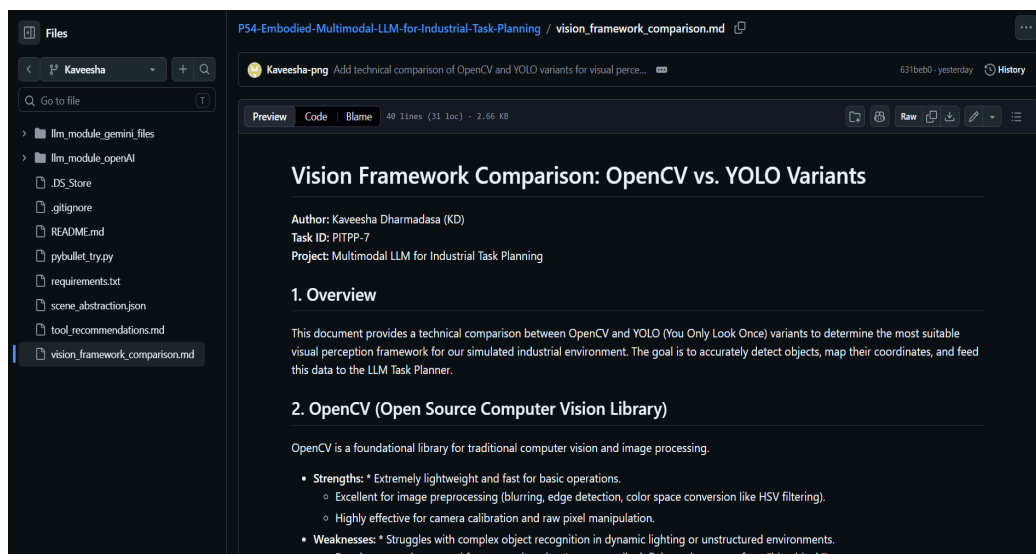
```
1 {
2   "scene_timestamp": "2026-04-13T10:00:00Z",
3   "workspace_status": "clear",
4   "detected_objects": [
5     {
6       "object_id": "obj_001",
7       "label": "block",
8       "attributes": {
9         "color": "blue",
10        "shape": "cube"
11      },
12      "position": {
13        "coordinates_3d": {"x": 0.45, "y": -0.20, "z": 0.05},
14        "bounding_box_2d": {"x_min": 120, "y_min": 340, "x_max": 180, "y_max": 400}
15      },
16      "spatial_relationships": [
17        {"relation": "on_top_of", "target_id": "obj_003"}
18      ],
19      "graspable": true
20    }
21  ]
22 }
```



3. Find and Compare Vision Frameworks (Task ID: PITPP-7)

I assumed the role of researching the visual perception module to also assess the best framework to use in our industrial simulation. I performed the detailed technical analysis of the traditional computer vision (OpenCV) and object detection based on deep learning (YOLO variants). I compared these two frameworks on parameters like computational load, bounding box accuracy and classification robustness in a dynamic environment. I saved these results in a comparative research markdown document (vision_framework_comparison.md) and added it to the repository, concluding that a hybrid YOLO-OpenCV strategy is obligatory to achieve precise abstraction of the scene with the help of LLM.

```
scene_abstraction.json  tool_recommendations.md  vision_framework_comparison.md X
C:\Users> cd > vision_framework_comparison.md > # Vision Framework Comparison: OpenCV vs. YOLO Variants
1 # Vision Framework Comparison: OpenCV vs. YOLO Variants
2 # 1. Overview
3 perception framework for our simulated industrial environment. The goal is to accurately detect objects, map their coordinates, and feed this
4 data to the LLM Task Planner.
5
6 # 2. OpenCV (Open Source Computer Vision Library)
7 OpenCV is a foundational library for traditional computer vision and image processing.
8
9 * **Strengths:** * Extremely lightweight and fast for basic operations.
10 * Excellent for image preprocessing (blurring, edge detection, color space conversion like HSV filtering).
11 * Highly effective for camera calibration and raw pixel manipulation.
12 * **Weaknesses:** * Struggles with complex object recognition in dynamic lighting or unstructured environments.
13 * Requires extensive manual feature engineering (e.g., manually defining color ranges for a "blue block").
14 * Not inherently designed for modern deep-learning-based classification without external models.
15
16 # 3. YOLO Variants (e.g., YOLOv8)
17 YOLO represents the state-of-the-art in real-time, deep-learning-based object detection.
18
19 * **Strengths:**
20 * Exceptional accuracy in detecting and classifying multiple objects simultaneously.
21 * Outputs highly precise 2D bounding boxes and class labels out-of-the-box.
22 * Robust against lighting changes, partial occlusions, and varied object orientations.
23 * **Weaknesses:**
24 * Computationally heavier than OpenCV (benefits significantly from GPU acceleration).
25 * Requires a trained dataset (though pre-trained models handle basic industrial objects well).
26
27 # 4. Comparative Summary & Conclusion
28
29 | Feature | OpenCV (Traditional) | YOLOv8 (Deep Learning) |
30 | --- | --- | --- |
31 | **Object Classification** | Low (Requires manual tuning) | High (Automated, robust) |
32 | **Bounding Box Accuracy** | Moderate (Contour-based) | Very High (Neural Network) |
33 | **Computational Load** | Very Low (CPU friendly) | High (GPU recommended) |
34 | **Setup Complexity** | High (Code-heavy logic) | Low (Plug-and-play inference) |
35
36 **Conclusion:** Relying solely on OpenCV is insufficient for the dynamic nature of intent-based robotic picking. However, YOLO alone lacks the
37 low-level camera feed manipulation required for simulation APIs. Therefore, a hybrid architecture is required: OpenCV should handle the camera
38 feed preprocessing and geometric coordinate mapping, while a YOLO variant (like YOLOv8) should be utilized for the core object detection and
39 classification pipeline.
```



TEAM MEMBER: DINITH THEJANA. (105231766)

During this sprint, I contributed to the **Vision Module development** by completing **PITPP-13, PITPP-15, and PITPP-16**.

For **Task PITPP-13 (Detected Object Output)**, I developed the `vision_output.py` module. In this task, I designed a structured JSON format to represent detected objects in the environment. Each object includes a **label, confidence score, and bounding box coordinates**. I created reusable functions such as `create_detection()` and `create_scene_output()` to ensure consistency and scalability. I tested this module using sample objects (red block, blue tray, green block) and successfully generated a valid `scene_output.json` file.

For **Task PITPP-15 (Scene Representation)**, I implemented the `scene_representation.py` module. This module processes the raw detection output and transforms it into a more meaningful structure. I calculated the **center point of each object** based on bounding box values and organised the data into a structured scene format. This step is important because it simplifies the data for further reasoning tasks. The output was saved as `scene_representation.json`.

For **Task PITPP-16 (Spatial Relationships)**, I developed the `spatial_relationships.py` module. In this task, I implemented logic to determine spatial relationships between objects, such as **left, right, and near**. I used object center coordinates to compare positions and applied **Euclidean distance** to identify nearby objects. This module generates human-readable relationships (e.g., “red block is left of blue tray”), which are useful for task planning and decision-making.

All three modules have been implemented within a basic pipeline where the result of one module acts as an input to the next module (Detection -> Scene Representation -> Spatial Relations). The complete implementation was done on my local system using VS Code and the results have been cross-verified for any errors in JSON logic or formatting.

Also, I was able to manage my task using GitHub by creating a separate branch in which I was able to commit my code without disrupting any other team member’s code.

Output: detected object list
with labels and bounding
boxes 

 PITPP-13  

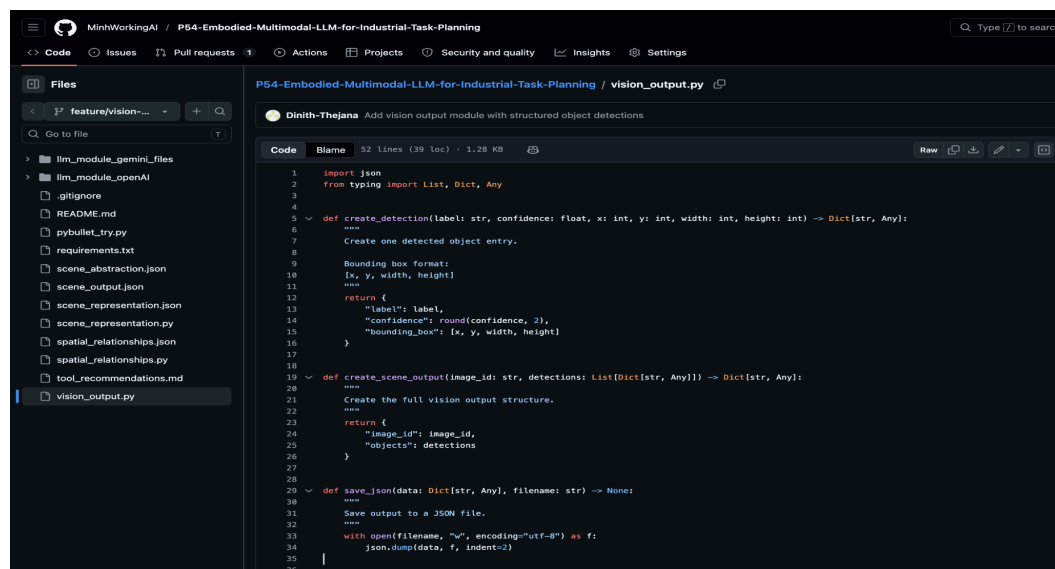
Convert vision module output
into structured scene
representation

 PITPP-15  

Handle spatial relationships
(left, right, near)

 PITPP-16  

Figure 1: tasks completed: PITPP-13, PITPP-15, PITPP-16

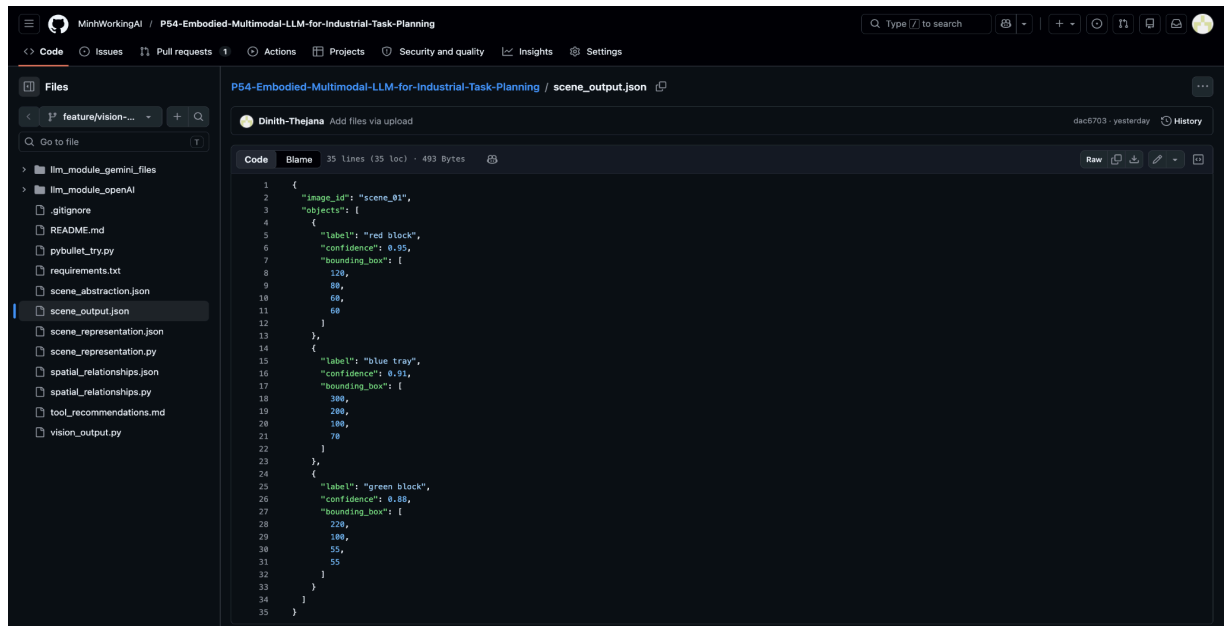


```

1  import json
2  from typing import List, Dict, Any
3
4
5  def create_detection(label: str, confidence: float, x: int, y: int, width: int, height: int) -> Dict[str, Any]:
6      """
7      Create one detected object entry.
8
9      Bounding box format:
10     [x, y, width, height]
11     """
12     return {
13         "label": label,
14         "confidence": round(confidence, 2),
15         "bounding_box": [x, y, width, height]
16     }
17
18
19 def create_scene_output(image_id: str, detections: List[Dict[str, Any]]) -> Dict[str, Any]:
20     """
21     Create the full vision output structure.
22     """
23     return {
24         "image_id": image_id,
25         "objects": detections
26     }
27
28
29 def save_json(data: Dict[str, Any], filename: str) -> None:
30     """
31     Save output to a JSON file.
32     """
33     with open(filename, "w", encoding="utf-8") as f:
34         json.dump(data, f, indent=2)
35
36

```

Figure 2: GitHub repository showing implementation of `vision_output.py` module with structured object detection output.



```
1 {
2   "image_id": "scene_01",
3   "objects": [
4     {
5       "label": "red block",
6       "confidence": 0.95,
7       "bounding_box": [
8         120,
9         80,
10        60,
11        60
12      ]
13    },
14    {
15      "label": "blue tray",
16      "confidence": 0.91,
17      "bounding_box": [
18        300,
19        200,
20        100,
21        70
22      ]
23    },
24    {
25      "label": "green block",
26      "confidence": 0.88,
27      "bounding_box": [
28        220,
29        100,
30        55,
31        55
32      ]
33    }
34  ]
35 }
```

Figure 3: JSON output (`scene_output.json`) showing detected objects with labels, confidence scores, and bounding boxes generated by the vision module

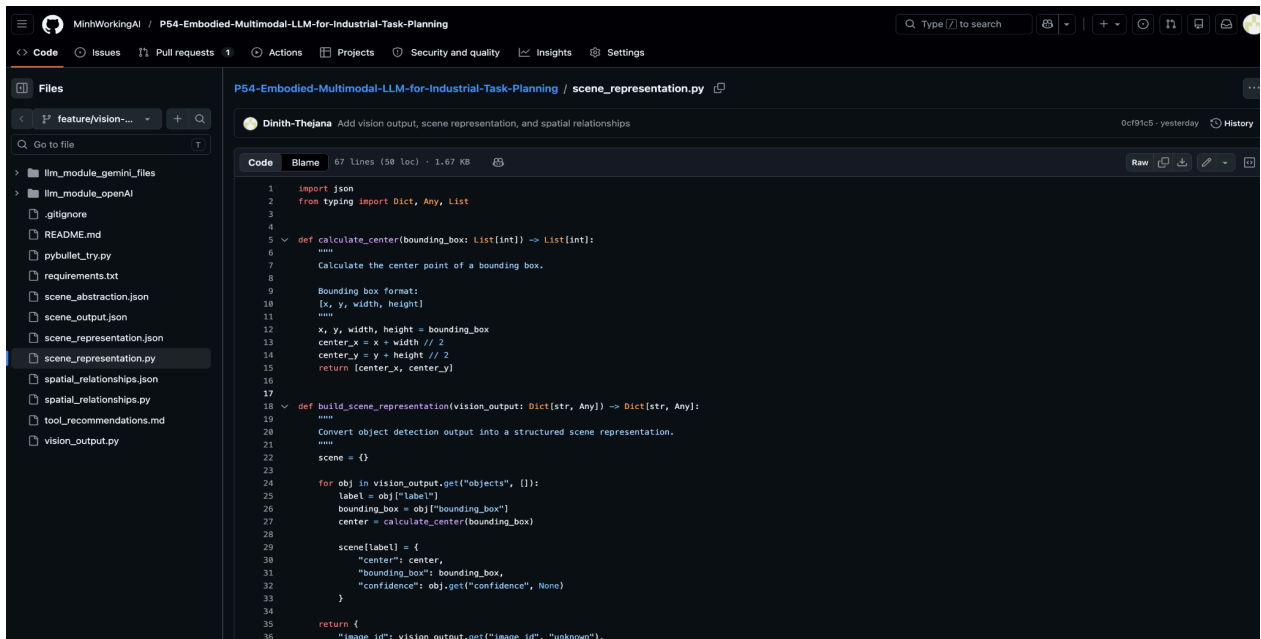


Figure 4: GitHub repository showing implementation of `scene_representation.py` module used to convert detection output into structured scene data

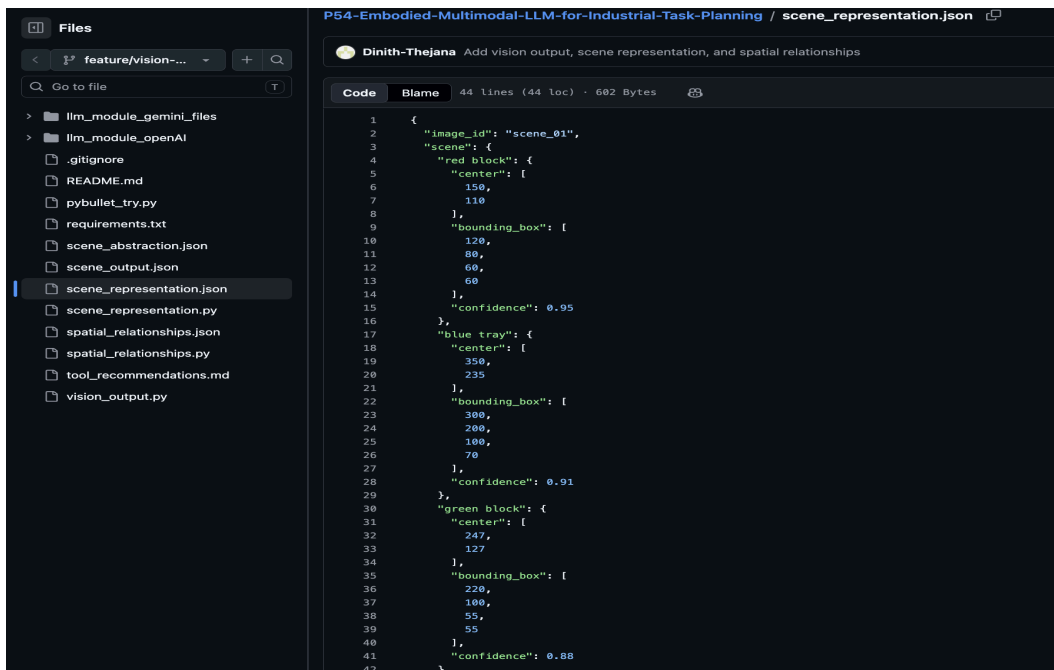


Figure 5: Structured scene representation (`scene_representation.json`) showing object centers, bounding boxes, and confidence values generated from the vision module output

```

1  import json
2  import math
3  from typing import Dict, Any, List
4
5  def load_json(filename: str) -> Dict[str, Any]:
6      """
7      Load JSON data from a file.
8      """
9      with open(filename, "r", encoding="utf-8") as f:
10         return json.load(f)
11
12
13
14  def save_json(data: Dict[str, Any], filename: str) -> None:
15      """
16      Save JSON data to a file.
17      """
18      with open(filename, "w", encoding="utf-8") as f:
19         json.dump(data, f, indent=2)
20
21
22  def euclidean_distance(point1: List[int], point2: List[int]) -> float:
23      """
24      Compute Euclidean distance between two center points.
25      """
26      return math.sqrt((point1[0] - point2[0]) ** 2 + (point1[1] - point2[1]) ** 2)
27
28
29  def compute_relationships(scene_data: Dict[str, Any], near_threshold: float = 100.0) -> List[str]:
30      """
31      Compute pairwise spatial relationships between objects.
32
33      Rules:
34      - If x1 < x2, object1 is left of object2
35      - If x1 > x2, object1 is right of object2
36      - If distance <= near_threshold, object1 is near object2
37      """
38      relationships = []
39      scene = scene_data.get("scene", {})
40      labels = List(scene.keys())
41

```

Figure 6: Implementation of spatial_relationships.py showing logic for computing left, right, and near relationships using object center coordinates

```

1  {
2      "image_id": "scene_01",
3      "relationships": [
4          "red block is left of blue tray",
5          "blue tray is right of red block",
6          "red block is left of green block",
7          "green block is right of red block",
8          "red block is near green block",
9          "green block is near red block",
10         "blue tray is right of green block",
11         "green block is left of blue tray",
12         "blue tray is near green block",
13         "green block is near blue tray"
14     ]
15 }

```

Figure 7: Spatial relationships output (spatial_relationships.json) showing positional relationships such as left, right, and near between detected objects

TEAM MEMBER: MINH HOANG DUONG

During Sprint 1, my primary contribution was taking ownership of the LLM backend refactoring and extension, building on the **llm_module** originally developed by Ved Jay Makhijani, and transforming it into a production-ready, multi-backend architecture. I also took responsibility for project tracking and coordination,

Task 1: LLM Backend Refactoring

After Sprint 1, I took ownership of refactoring and significantly extending the **llm_module** originally built by Ved Jay Makhijani into a production-ready, multi-backend architecture renamed **llm_backend**.

The original codebase had a fundamental structural problem: instead of a single modular LLM module, the team had accumulated two separate folders — **llm_module_gemini_files** and **llm_module_openAI** — each containing near-identical copies of the same files (**parser.py**, **schema.py**, **prompts.py**, **edge_cases.py**, **hello_world.py**, **demo.py**, **__init__.py**). The only meaningful difference between the two folders was which API client was imported at the top of **parser.py** and which environment variable was read for the API key.

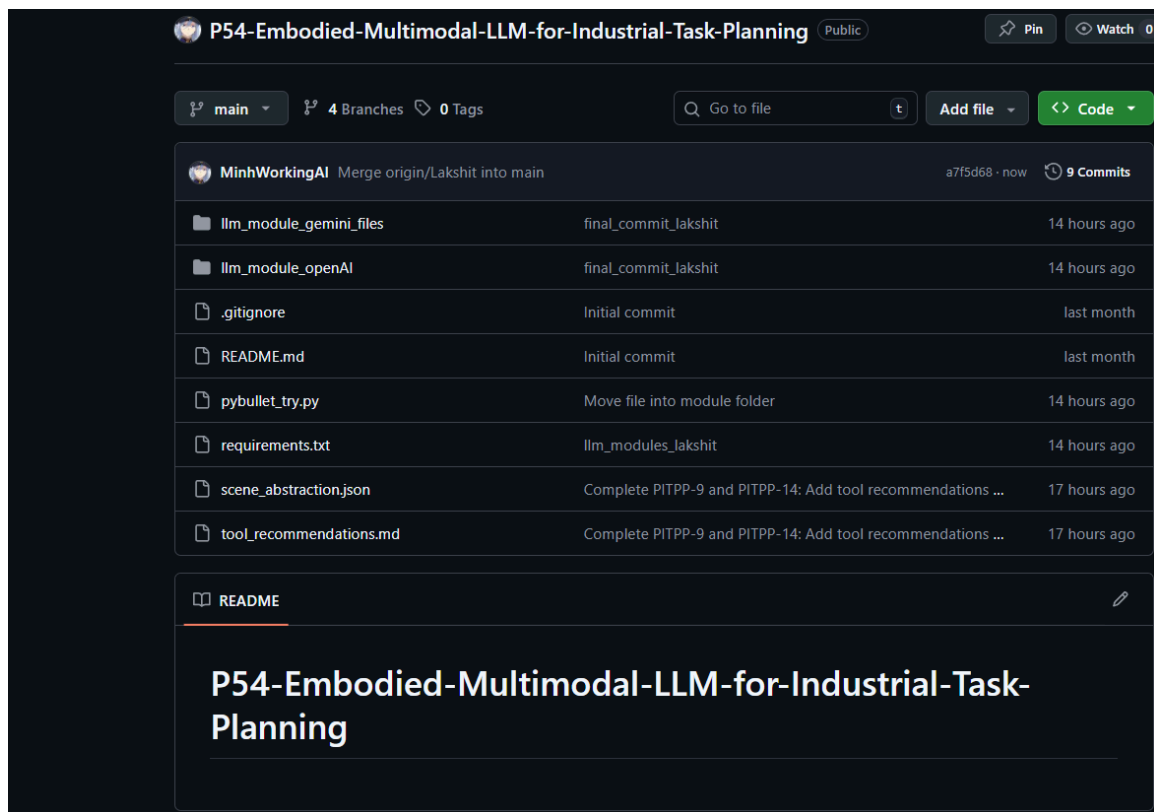


Figure 1.0: LLM backend before refactoring.

I resolved all of the above by consolidating everything into a single **llm_backend** folder with a clean separation between shared logic and backend-specific code:

- All backend-agnostic files (**schema.py, prompts.py, edge_cases.py, custom_LLM_parser.py, demo.py, hello_world.py, test_llm_module.py**) exist exactly once
- All backend-specific code is isolated to individual files under backends/ — one file per backend, each owning its own credentials and model configuration exclusively
- A backends/__init__.py factory reads a single environment variable (LLM_BACKEND) and routes to the correct backend at runtime with no code changes required.

The **llm_module_gemini_files** and **llm_module_openAI** folders were deleted from the repository. The full refactored structure is:

llm_backend/

```
|— __init__.py
|— custom_LLM_parser.py    -- single parser, works with all backends
|— schema.py              -- one schema, shared by everything
|— prompts.py             -- one prompt, shared by everything
|— edge_cases.py          -- one edge case handler, shared by everything
|— backends/
|   |— __init__.py        -- get_llm() factory
|   |— openai_backend.py  -- OpenAI credentials and init only
|   |— gemini_backend.py  -- Gemini credentials and init only
|   |— deepseek_backend.py -- DeepSeek credentials and init only
|   |— huggingface_backend.py -- local inference, no credentials needed
|— demo.py
|— hello_world.py
|— test_llm_module.py
|— requirements.txt
|— .env.example
```

Adding a new backend in the future (e.g. Anthropic Claude) requires only creating one new file under **backends/** and adding one elif line to **backends/__init__.py**. No other file needs to change.

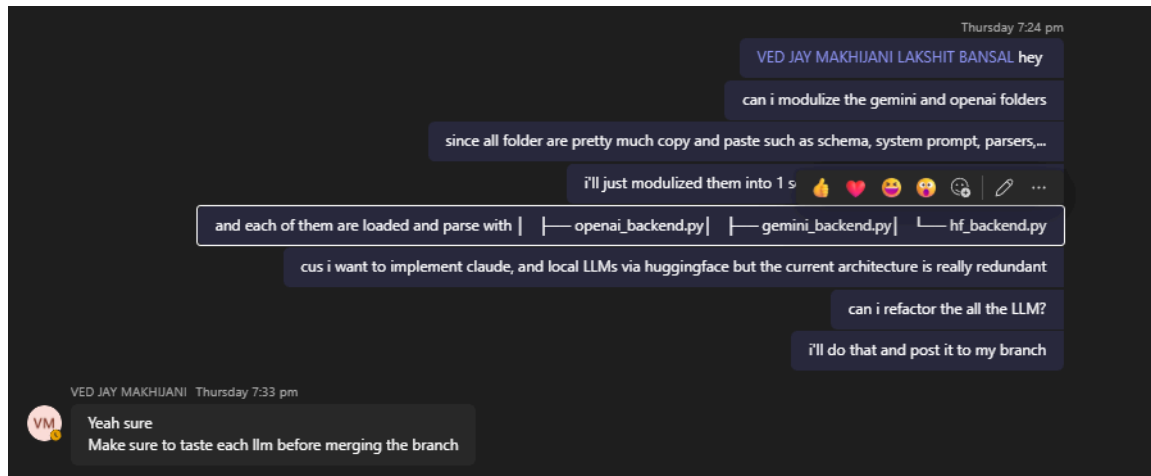


Figure 2.0: Refactoring LLM backend permission from Ved Jay

Furthermore, I implemented 2 extra new backends in LLM:

- **openai_backend.py** — ChatOpenAI via LangChain, reads **OPENAI_API_KEY**, **OPENAI_MODEL**, **OPENAI_TEMPERATURE**, implemented based on Ved Jay creation.
- **gemini_backend.py** — ChatGoogleGenerativeAI, reads **GEMINI_API_KEY**, **GEMINI_MODEL**, **GEMINI_TEMPERATURE**, implemented based on Ved Jay creation.
- **deepseek_backend.py** — native ChatDeepSeek from langchain-deepseek, reads **DEEPSEEK_API_KEY**, **DEEPSEEK_MODEL**, **DEEPSEEK_TEMPERATURE**, newly added.
- **huggingface_backend.py** — fully local inference using **HuggingFacePipeline** + **ChatHuggingFace**, no API key required. Supports **DeepSeek R1 Distill**, **Qwen 3**, **Qwen 2.5**, **Llama 3**, **Mistral**, **Gemma 3**, and **Phi-4** families. Includes automatic CPU/GPU detection (float16 on GPU, float32 on CPU, newly added).
- Switching backend requires only a single line change in **.env**:

I also Created **requirements.txt** with clear install instructions including the mandatory manual PyTorch CUDA install step. Created **.env.example** documenting all four backends with recommended model options by RAM tier for the HuggingFace backend.

```
.env.example X
PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning > .env.example
1 # LLM_BACKEND=openai/gemini/deepseek/huggingface
2
3 # OPENAI_API_KEY=your_openai_api_key_here
4 # OPENAI_MODEL=gpt-5-nano-2025-08-07
5 # OPENAI_TEMPERATURE=0.0
6
7 # GEMINI_API_KEY=your_gemini_api_key_here
8 # GEMINI_MODEL=gemini-2.0-flash
9 # GEMINI_TEMPERATURE=0.0
10
11 # Deepseek_API_KEY=your_deepseek_api_key_here
12 # Deepseek_MODEL=deepseek-chat
13 # Deepseek_TEMPERATURE=0.0
14
15 # HF_MODEL=Qwen/Qwen2.5-7B-Instruct
16 # HF_DEVICE=auto
17 # HF_MAX_TOKENS=512
18 # HF_LOAD_IN_4BIT=false # only set to true if you have GPU and want to save VRAM, but it may reduce performance
19 # HF_TOKEN=your-huggingface-token # only needed for gated models (Llama, Gemma)
20
21
22 _
```

Figure 3.0: .env.example

```
requirements.txt X
PS4-Embodied-Multimodal-LLM-for-Industrial-Task-Planning > requirements.txt
1 # LLM Backend
2 gptai>1.0.0
3 langchain>0.1.0
4 langchain-openai>0.0.5
5 langchain-google-gemini>0.0.5
6 langchain-deepseek>0.1.0
7 langchain-core>0.2.0
8
9 #Huggingface local LLM
10 langchain-huggingface>0.1.0
11 transformers>4.40.0
12 accelerate>0.30.0
13 bitsandbytes>0.43.0 # 4-bit quantization (optional, GPU only)
14
15 #Note: Torch is a dependency of transformers, so you might need to uninstall the existing version and install the compatible one
16 # pip uninstall torch torchvision torchaudio -y
17 #
18 # CUDA 12.1 (most common):
19 # pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
20 #
21 # CUDA 11.8:
22 # pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
23 # CPU only (no GPU):
24 # pip install torch torchvision torchaudio
25 #
26 # Best practice is to manually install the compatible torch version first, then install the rest of the dependencies, to avoid version conflicts.
27 # pip install -r requirements.txt
28
29
30 # Computer Vision
31 opencv-python>4.8.0
32 ultralytics>8.0.0
33 numpy>1.24.0
34 Pillow>10.0.0
35
36 # Simulation
37 pybullet>3.2.5
38 pygame>2.5.0
39
40 # Utilities
41 python-dotenv>1.0.0
42 pydantic>2.0.0
43
44 # Testing
45 pytest>7.4.0
```

Figure 4.0: requirements.txt with installation note.

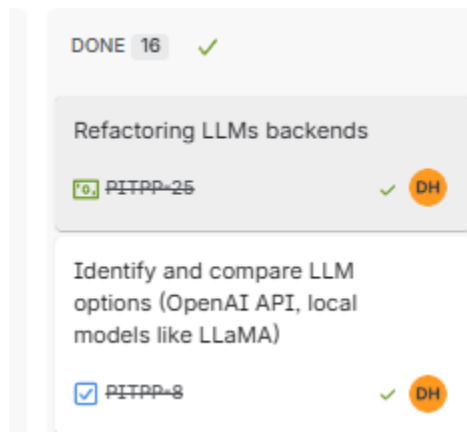


Figure 5.0: Minh Hoang Duong Jira Ticket

TEAM MEMBER: LAKSHIT BANSAL (105028858)

During Sprint 1, my primary contribution was the **development of the PyBullet simulation environment and integration of computer vision with the system**, along with **debugging platform compatibility issues and extending the LLM module to support Gemini API**.

Task 1: PyBullet Simulation Setup and Debugging (Robotics & Simulation)

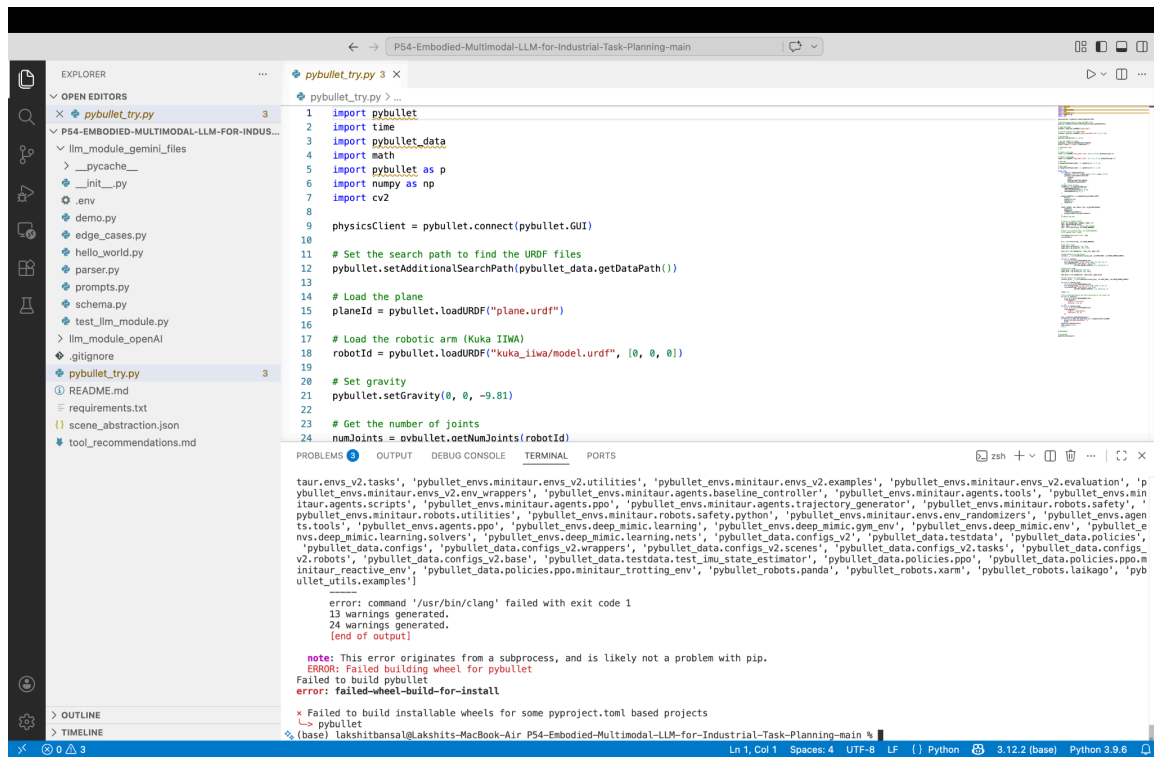
I was responsible for setting up the **PyBullet simulation environment** from scratch. This included loading the physics world, configuring the robotic setup, and ensuring that the simulation loop runs correctly.

A major challenge encountered during this process was **installing and running PyBullet on macOS devices with ARM (Apple Silicon) chips**. Through extensive research and debugging, I identified that PyBullet has **limited or unstable support for ARM-based macOS systems**, which caused installation failures and runtime issues.

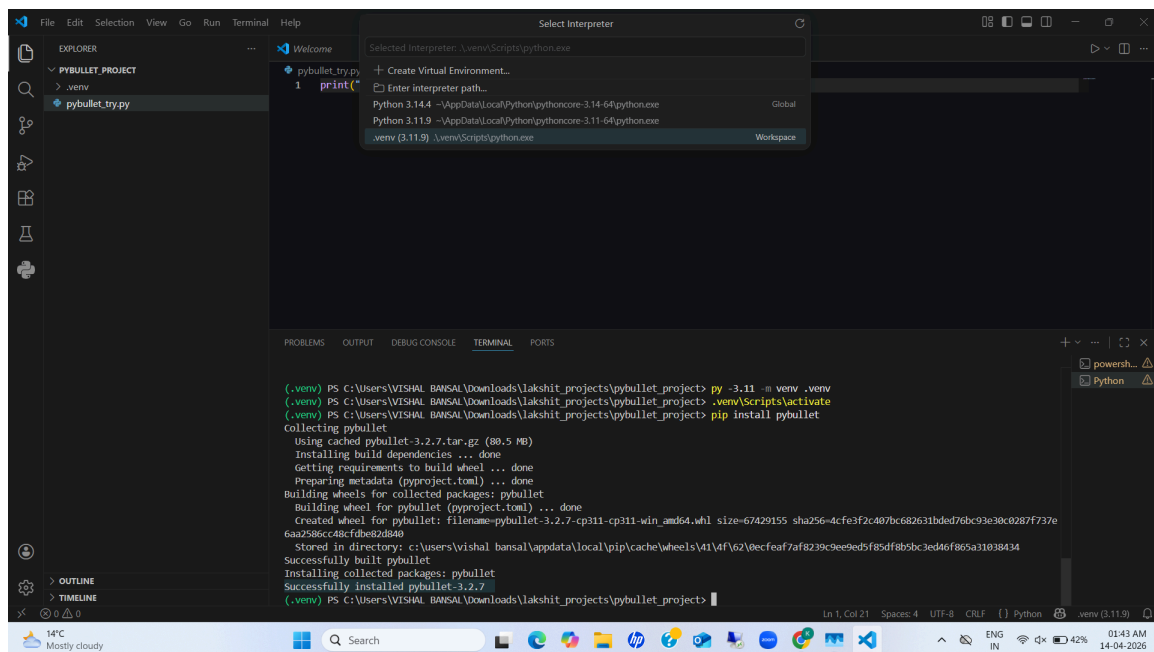
To address this:

- I tested multiple installation methods (pip, source builds, dependency fixes)
- Investigated compatibility issues with Python versions and system architecture
- Documented that the simulation works reliably only on supported environments (e.g., x86 systems)

This finding was important for the team, as it directly impacted testing and development workflow across different devices.



Screenshot 1: PyBullet installation errors / debugging process on mac



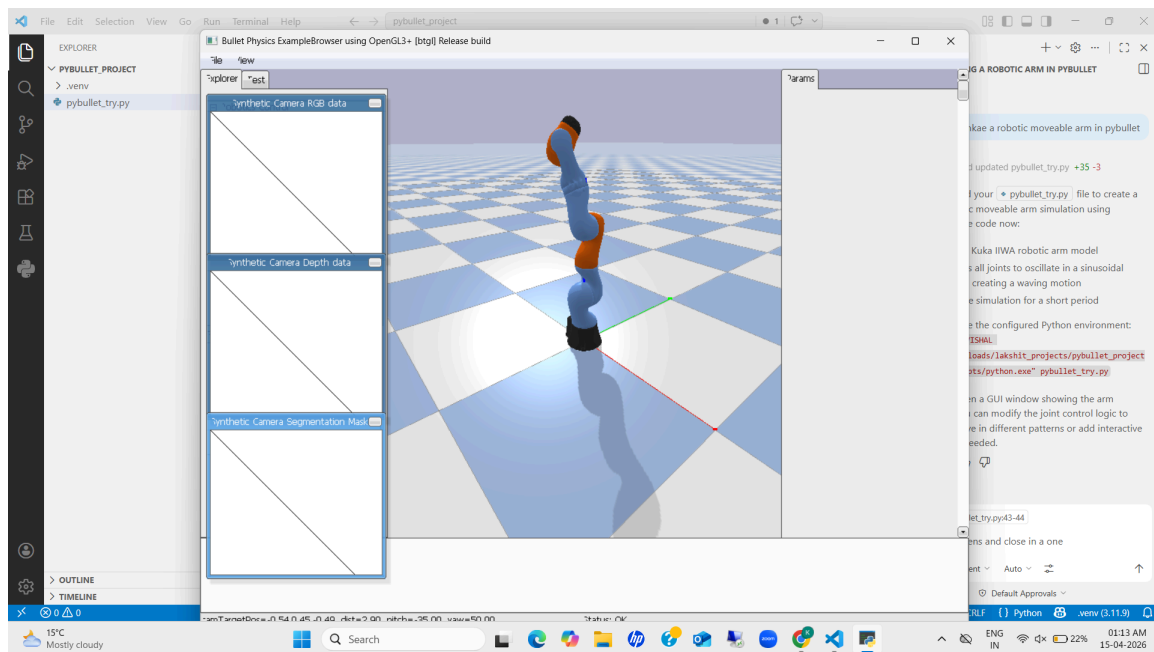
Screenshot 2: Working simulation environment on Windows

Task 2: Simulation Environment Development

I developed the full simulation environment, including:

- Loading the robotic manipulator (KUKA arm)
- Creating the workspace with objects (e.g., coloured blocks)
- Running the physics simulation loop at real-time speed
- Ensuring stable execution and interaction within the environment

The simulation was designed to act as the **foundation for integrating perception and planning modules**.



Screenshot 3: Simulation environment with robot

Task 3: Computer Vision Integration with OpenCV

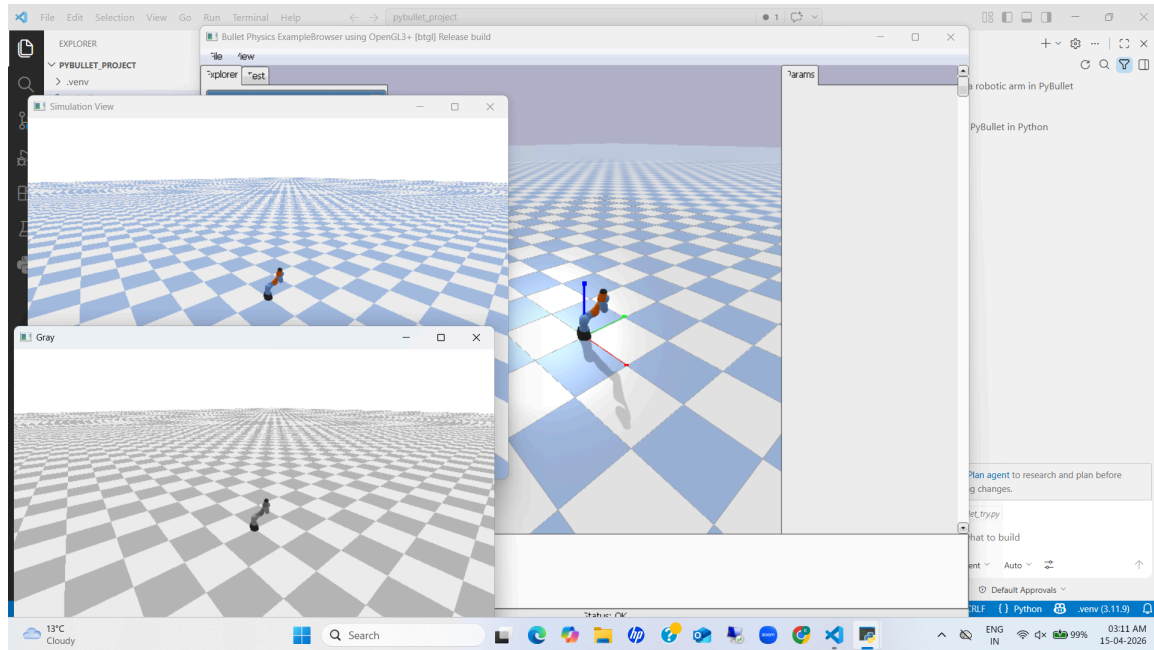
I extended the simulation by integrating **OpenCV-based computer vision** to enable object detection within the environment.

This involved:

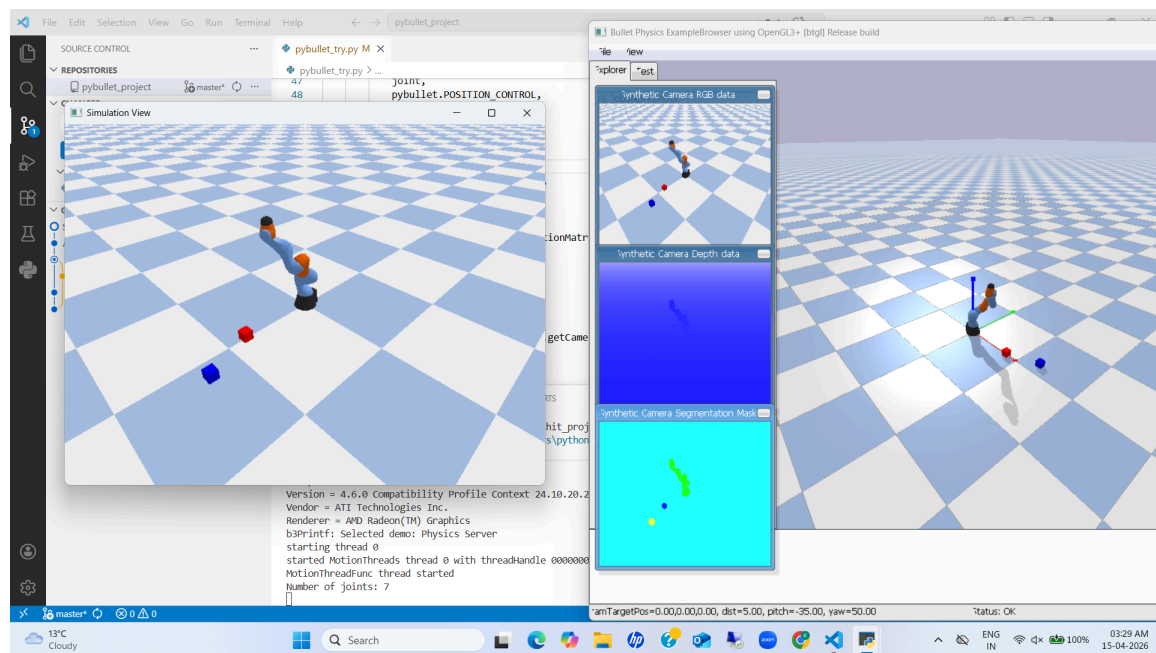
- Capturing camera feed from the PyBullet simulation
- Processing frames using OpenCV
- Detecting objects based on colour (HSV masking)
- Extracting object positions and bounding boxes

- Mapping detected objects into usable data for further processing

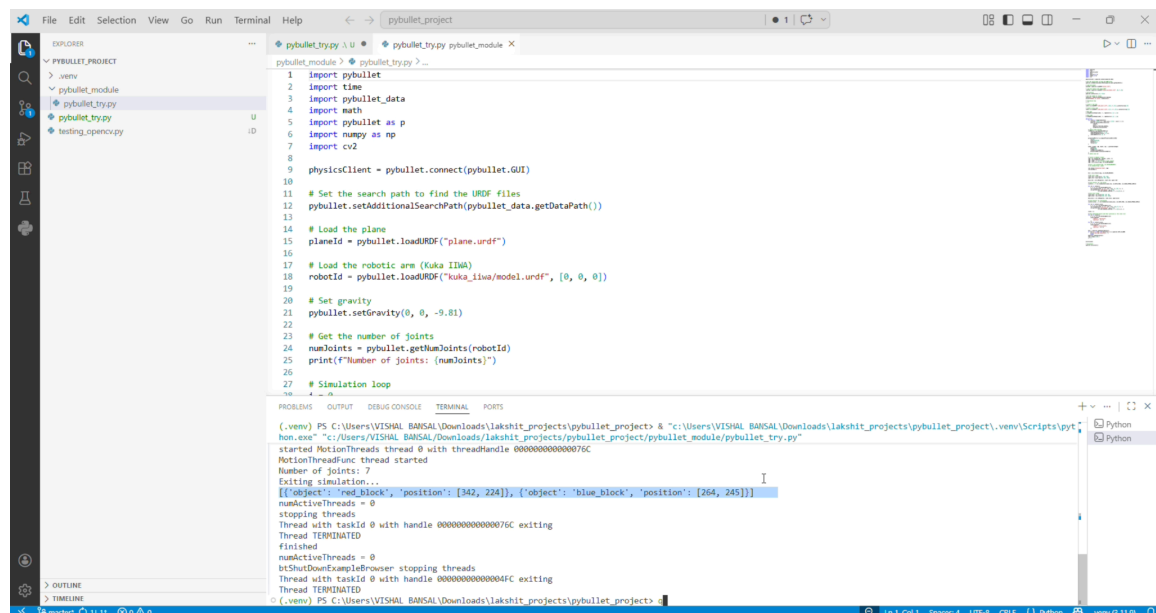
This step was critical in bridging the gap between the **simulation environment and perception pipeline**.



Screenshot 4: OpenCV visualization of the environment



Screenshot 5: OpenCV detection with bounding boxes



Screenshot 6: OpenCV detection of positions of the objects

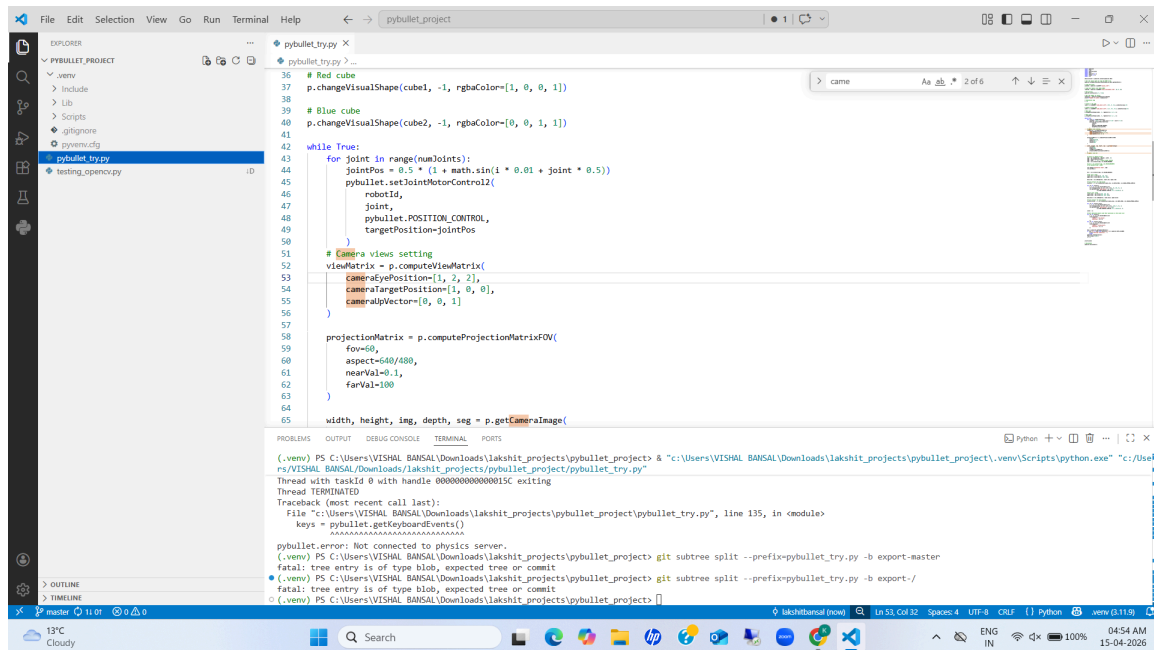
Task 4: Camera Configuration and Optimization

To improve detection accuracy, I experimented with different camera positions and angles within the simulation.

This included:

- Adjusting camera placement to maximize visibility of objects
- Testing multiple viewpoints
- Evaluating detection quality based on perspective

These adjustments helped identify the optimal camera configuration for reliable object detection.



Screenshot 5: Different camera views / configurations

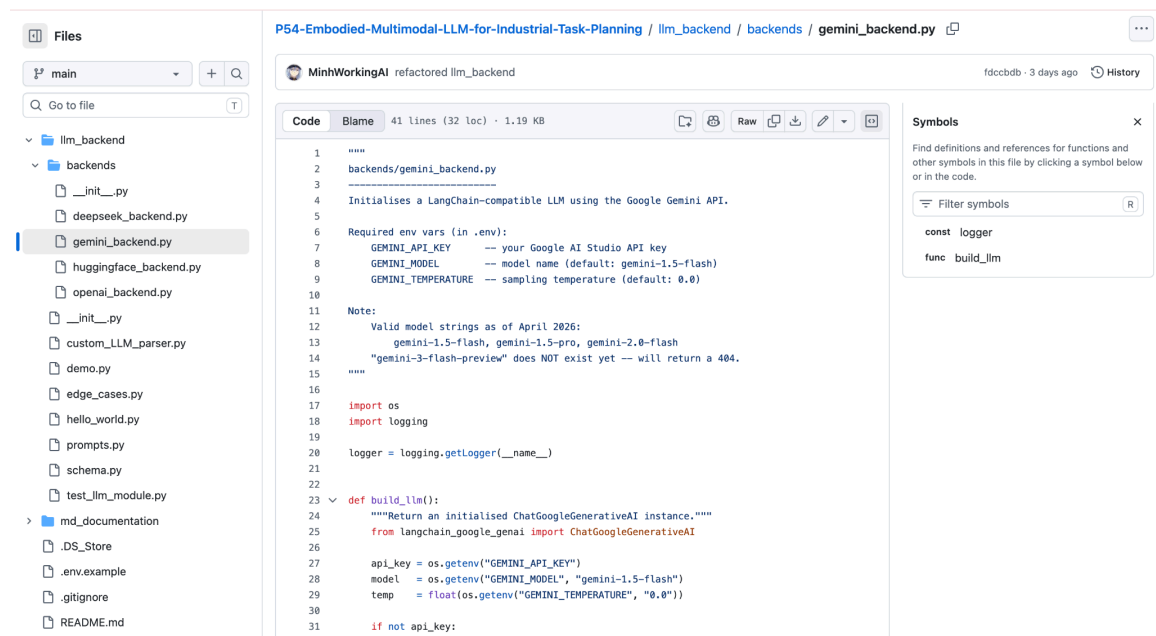
Task 5: LLM Module Enhancement (Gemini API Integration)

In addition to simulation work, I contributed to the LLM module by enabling support for the Gemini API, allowing the system to use a free alternative to OpenAI APIs.

This involved:

- Modifying the LLM module to accept Gemini API keys
- Ensuring compatibility with existing parsing logic
- Testing responses to confirm correct functionality

This enhancement improves the accessibility and cost-efficiency of the system, making it easier for team members to test without relying on paid APIs.



Screenshot 6: Gemini API integration

Summary of Contribution

Overall, my contributions enabled the project to have:

- A fully functional simulation environment
- Integrated computer vision pipeline
- Optimized camera setup for detection
- Documented platform limitations (macOS ARM issue)
- Extended LLM support with Gemini API

These components form a critical part of the system, connecting the simulation, perception, and AI modules into a working prototype.

Task 2: Project Tracking and Sprint Documentation

I maintained the Jira board throughout the sprint, tracking ticket status for all team members and flagging blockers during standup meetings. I coordinated the sprint review preparation and compiled the sprint report including the contribution summary table, sprint plan, progress section, sprint review, retrospective, and lessons learned sections.

1. SPRINT PLAN

Sprint Goal: The overall aim of Sprint 1 was to develop the underlying software architecture and provide the functional, standalone prototypes of the two main pillars of the pipeline: the LLM Natural Language Parser and the PyBullet 3D Simulation environment.

Sprint Capacity: 5 Team members.

Sprint Backlog & Estimated Effort (By Epics):

Epic 1: System Architecture & Data Flow

- * **Repository Setup (2 Hours):** Create the GitHub repository, create the Python virtual environment, and configure the modular folder structure.
- * **Vision Framework Research (PITPP-7) (3 Hours):** A technical comparison of OpenCV and YOLOv8 to complete the visual perception architecture.
- * **Scene Abstraction Data Structure (PITPP-14) (4 Hours):** Invent a standardized JSON schema to become the rigid communication contract between the Vision module and the LLM Task Planner.

Epic 2: Natural Language Processing (LLM Module).

- * **API Integration (3 Hours):** Establish and implement secure integrations with OpenAI (GPT-4) and Google Gemini API to compare them.
- * **Prompt Engineering & Parsing (PITPP-19) (5 Hours):** Implement the business logic to read unstructured natural language instructions and extract the structured intents (Action, Object, Spatial Relations).
- * **Edge Case Handling (PITPP-21) (3 Hours):** Add unit tests (e.g., demo.py, edge_cases.py) to verify the ability to robustly interpret ambiguous instructions and avoid hallucinated arguments.

Epic 3: Robotics & Simulation

- * **Start PyBullet Environment (6 Hours):** Load the simplest 3D physics world, including the robotic manipulator and objects of interest (e.g., blocks, trays) on the workspace.
- * **Configure Simulated Sensors (4 Hours):** Configure virtual synthetic cameras to the simulation to effectively produce RGB data, Depth data and Segmentation Masks to be used in future vision processing.

Definition of Done (DoD) of this Sprint:

- Code and documentation are committed and merged to the main GitHub branch successfully.

- Without any fatal runtime errors, the LLM script can accept a string input and print a formatted JSON output.
- The simulation is successful with the robot and synthetic camera feeds visible and active.

2. SPRINT PROGRESS

Sprint 1 achieved its primary goal of delivering functional standalone prototypes for both the LLM Natural Language Parser and the PyBullet simulation environment.

- **LLM Module (Ved Jay Makhijani):** The full instruction parser was completed end-to-end. The module accepts natural language input and returns structured JSON conforming to the ParsedInstruction schema. All 24 unit tests pass without API credits (pytest -m "not integration").

The interactive demo (python demo.py --interactive) was verified live against the OpenAI API, correctly handling simple, spatial, synonym-mapped, and ambiguous instructions.

- **Vision Framework Research & Tool Selection (Kaveesha Dharmadasa):** A detailed technical comparison of OpenCV and YOLOv8 was completed and documented in vision_framework_comparison.md.

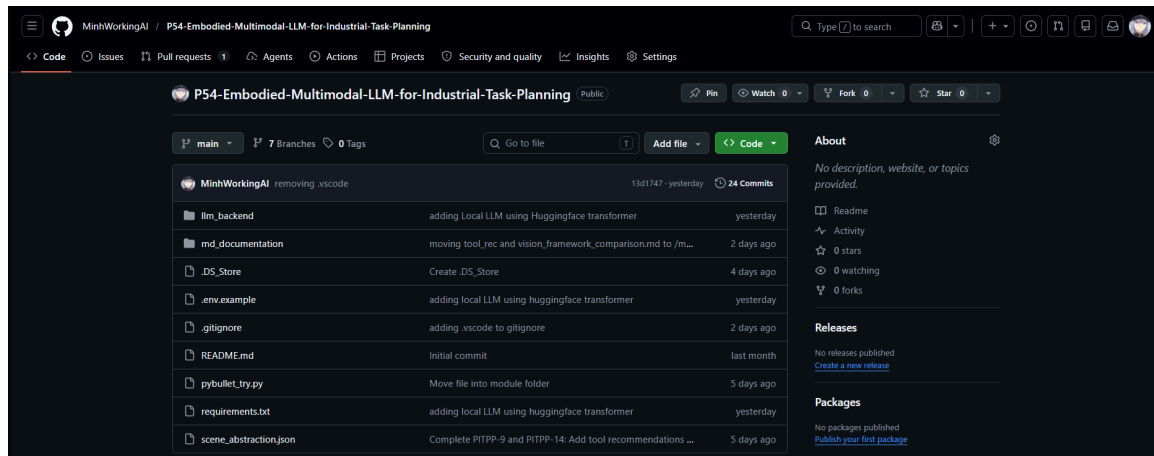
Architectural recommendations were finalised in tool_recommendations.md, concluding that a hybrid YOLOv8 + OpenCV strategy is required.

The Scene Abstraction JSON schema (scene_abstraction.json) was designed as the formal interface contract between the Vision module and the LLM Task Planner, defining object IDs, 3D coordinates, bounding boxes, and spatial relationships.

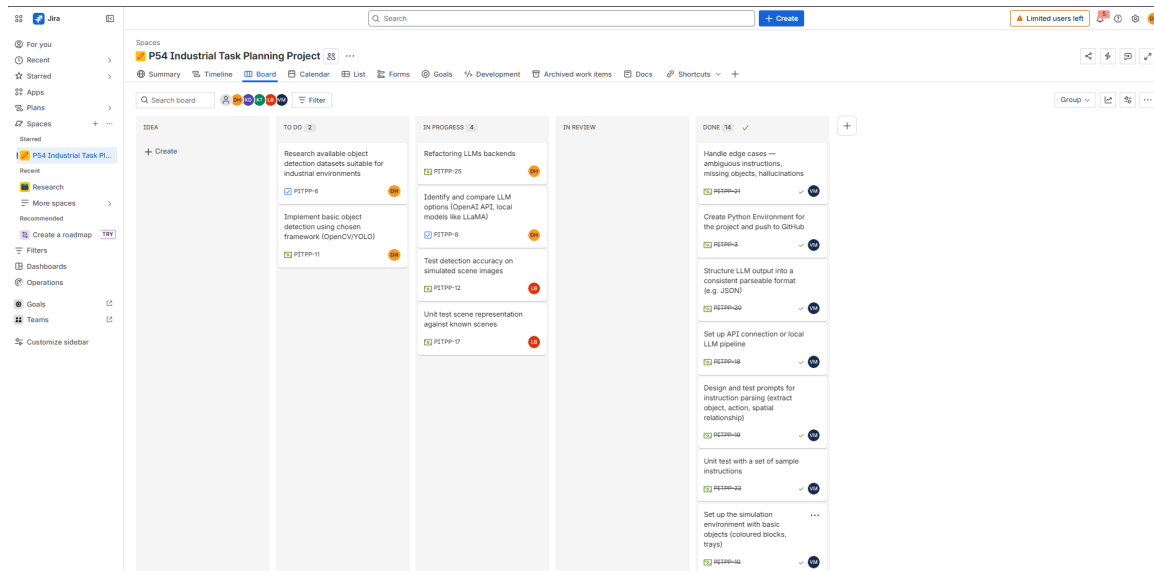
- **Vision Pipeline (Dinith Thejana):** Three vision modules were implemented and committed on a dedicated branch. vision_output.py produces structured detected object JSON with labels, confidence scores, and bounding boxes. scene_representation.py converts raw detections into a centroid-based scene format. spatial_relationships.py computes left, right, and near relationships using Euclidean distance between object centers. All three modules form a working pipeline (Detection → Scene → Spatial Relations) with verified JSON outputs.
-
- **PyBullet Simulation (Lakshit Bansal):** The PyBullet environment was initialised with a Kuka IIWA robotic arm, a ground plane, and two coloured blocks (red and blue). A virtual camera was configured to produce RGB frames. OpenCV HSV masking was applied to detect block colours and draw bounding boxes in real time. The simulation loop runs at 240Hz with keyboard interrupt (q) for clean exit.

Evidence:

[P54-Embodied-Multimodal-LLM-for-Industrial-Task-Planning](#): Github Repository shows commits from all members across respective branches.



[P54 Industrial Task Planning Project](#): the jira board reflects all Sprint 1 tickets.



3. SPRINT REVIEW

Demonstration Session

The Sprint 1 demonstration was conducted with the supervisor and client. Lakshit Bansal and Ved Jay Makhijani represented the team, presenting two core deliverables: the PyBullet simulation environment running with a KUKA arm and OpenCV object detection, and the LLM instruction parser demonstrated live via the interactive demo script.

Client Feedback

The client received the Sprint 1 increment positively and accepted it as deliverable. Key feedback included:

- The LLM module's structured output schema and test coverage were highlighted as a strong foundation. The client noted that having a formal interface contract (ParsedInstruction schema) agreed upon by the team early was the right approach for a modular system of this complexity.
- The client reviewed the comparative LLM evaluation and noted that benchmarking multiple models against concrete metrics was the right approach for justifying model selection in a system with real-time requirements. The evaluation table comparing GPT-4o, DeepSeek, and Gemini across instruction accuracy, latency, parse success rate, and error rate was accepted as a sufficient response to the client's earlier request to validate the model choice with evidence. The client confirmed that GPT-4o's combination of lowest latency (1,149.6ms average) and zero error rate made it the appropriate primary model for the current pipeline, and considered this evaluation complete and deliverable for Sprint 1.
- The client acknowledged the macOS ARM compatibility issue with PyBullet as an environmental constraint rather than a design flaw, but flagged that the team must ensure all members can run and test the simulation locally before Sprint 2 integration begins — development bottlenecks from platform incompatibility are not acceptable as a recurring issue.
- Overall, the client considered the progress reflective of a sufficient team contribution for Sprint 1, given that both primary epics (NLP and simulation) produced functional, demonstrable prototypes.

Critical Analysis of Progress and Feedback

The client's acceptance of the increment as deliverable validates the team's decision to prioritise functional, standalone prototypes over premature integration in Sprint 1. The LLM module in

particular exceeded baseline expectations — the 24-unit-test suite and multi-backend architecture provide a level of robustness that positions Sprint 2 integration work on a stable foundation.

However, the client's feedback on the sensor pipeline reveals a gap between what was planned and what was delivered. Epic 3 was only partially completed: the simulated camera feed is running but the depth and segmentation mask output — which is required for the vision modules to move beyond colour-based detection — was not implemented. This is a meaningful shortfall because Dinith's vision pipeline (scene representation and spatial relationships) currently operates on simulated detection data rather than real sensor output from the simulation. The two halves of the system are not yet connected in a live pipeline.

The late finalisation of the Scene Abstraction JSON schema also caused rework on the vision side, as noted by Kaveesha and Dinith. The client's implicit expectation is that by Sprint 2, the interface between simulation, perception, and planning is live and demonstrable end-to-end — not just module-by-module. This sets a clear and demanding target for the next sprint.

4. RETROSPECT (CRITICAL REVIEW OF THE PROCESS)

Sprint Process Review

The team adopted an Agile methodology for Sprint 1, using Jira for task tracking and GitHub for version control. The following is a critical review of the process across key dimensions.

Team Plan

The sprint plan was well-structured at the epic level, with clear task ownership and time estimates. However, task dependencies were not explicitly mapped — modules were developed in parallel without a formal handoff sequence. This led to the Scene Abstraction JSON schema being finalised mid-sprint rather than on day one, which caused partial rework on the vision pipeline. In future sprints, the team plan must identify which tasks are blocking dependencies and sequence them accordingly before sprint work begins.

Communication Plan

The team communicated primarily via Microsoft Teams and weekly meetings, which was sufficient for status updates. However, the communication plan was informal and verbal — there was no documented agreement on response time expectations, escalation paths for blockers, or a defined standup format. In practice, some members were blocked waiting for schema decisions that were only being discussed synchronously in meetings, causing avoidable delays. A documented communication plan with defined async update expectations (e.g., daily status post by end of day) would have surfaced these blockers earlier.

Quality Plan

There was no formally documented quality plan for Sprint 1. In practice, quality varied across modules: the LLM module had a comprehensive 24-test suite with clear separation of unit and integration tests, while other modules (vision pipeline, simulation) had no equivalent automated testing. Code review before merging to main was also inconsistent, with some members committing directly to branches without peer review. Sprint 2 should establish a minimum quality standard that applies to all modules — at minimum, a passing set of unit tests and a peer-reviewed pull request before any code is merged to main.

Risk Registry

The team did not maintain a formal risk registry during Sprint 1. Two significant risks materialised that were not anticipated in planning: the macOS ARM incompatibility with PyBullet, which prevented two team members from running the simulation locally, and the late-stage finalisation of the shared data schema, which created integration rework. Had these been identified as risks upfront — platform compatibility as a high-likelihood risk for any native physics library, and interface contract agreement as a dependency risk, mitigation steps could have been planned before they caused delays. Sprint 2 must include a maintained risk registry with likelihood, impact, and mitigation owner recorded for each identified risk.

Ethical Considerations

The team identified two relevant ethical considerations during Sprint 1. First, all LLM API calls currently route through third-party providers (OpenAI, Gemini, DeepSeek), which raises data handling concerns under the Australian Privacy Principles (APPs) if the system is eventually deployed with real factory instructions containing sensitive operational data. The team has identified local model inference via HuggingFace as a mitigation path, but this has not yet been validated as a production-ready fallback. Second, in a safety-critical industrial robotics context, LLM hallucination poses a direct physical risk — a misclassified action type or incorrect destination field in the ParsedInstruction output could result in the robot executing an unintended movement. The current edge case handling and confidence scoring partially address this, but a formal output validation layer with a reject-and-request-clarification path for low-confidence outputs should be prioritised before any real-world deployment is considered.

5. LESSONS LEARNED (CRITICAL REVIEW OF SPRINT ONE EXPERIENCE AND FUTURE PLAN)

- **Define interface contracts before coding begins.** The Scene Abstraction JSON schema was designed mid-sprint, which caused the vision pipeline modules to be partially reworked once the format was finalised. In Sprint 2, all shared data formats must be agreed upon and committed to the repository on day one before any module implementation begins.
- **Test across all team machines early.** PyBullet failed on macOS devices used by two team members. Hardware and OS compatibility checks should be part of the definition of done for any environment setup task.
- **Separate API-dependent tests from unit tests from the start.** The `@pytest.mark.integration` pattern used in the LLM module proved valuable -- team

- members without API keys could still verify all core logic. This pattern should be adopted across all modules in Sprint 2.
- **Branch naming and merge strategy must be agreed upfront.** Git branching conflicts and authentication issues caused delays during integration. The team will adopt a consistent branch naming convention (**feature/module-name**) and require pull requests to be reviewed before merging to main.
 - **Local model fallback is not optional.** The Australian Privacy Principles requirement for sensitive factory data means a local LLM backend cannot be treated as a future nice-to-have. Sprint 2 should include at minimum a working local inference option, even if it is only used for testing privacy-sensitive scenarios.
 - **Communication of module status should be async.** Some team members were blocked waiting for schema decisions that were being discussed in meetings.